用 CODEX 构建 · APP SERVER

Codex App Server

通过 app-server 协议将 Codex 嵌入你的产品

译自 learn.chatgpt.com/docs/app-server（OpenAI 官方文档，中文翻译）

> 完整的文档索引请参见 llms.txt。在页面 URL 末尾追加 `.md` 即可获取文档页面的 Markdown 版本。

Codex app-server 是 Codex 用来驱动富客户端（例如 Codex VS Code 扩展）的接口。当你希望在自己的产品中实现深度集成——身份认证、对话历史、审批以及流式传输的智能体（agent）事件——时，可以使用它。app-server 的实现已在 Codex GitHub 仓库中开源（openai/codex/codex-rs/app-server）。完整的 Codex 开源组件列表请参见 Open Source 页面。

> ⓘ 如果你要自动化任务或在 CI 中运行 Codex，请改用 Codex SDK。

连接 CLI 终端 UI

远程终端 UI 模式让你可以在一台机器上运行 app-server，并从另一台机器连接 Codex CLI 终端界面。启动一个 WebSocket 监听器：

```
codex app-server --listen ws://127.0.0.1:4500
```

然后连接终端 UI：

```
codex --remote ws://127.0.0.1:4500
```

对于非本地连接，请配置 WebSocket 身份认证，并将连接置于 TLS 保护之下。将 bearer token 存储在环境变量中并传递其名称， 而不是把 token 放在命令行上：

```
export CODEX_REMOTE_TOKEN="$(cat "$HOME/.codex/app-server-token")"
codex --remote wss://remote-host:4500 \
  --remote-auth-token-env CODEX_REMOTE_TOKEN
```

`--remote` 选项接受 `ws://`、`wss://`、`unix://` 和 `unix://PATH` 端点。仅在 localhost 或 SSH 端口转发连接上使用普通 WebSocket。

连接远程 Code Mode 主机

默认情况下，app-server 会启动一个本地 Code Mode 主机。若要在同一 app-server 进程内的多个线程（thread）之间共享一台远程主机，请把远程主机的 HTTP/2 gRPC 端点传给 `--code-mode-host`。

```
codex app-server --code-mode-host https://code-mode.example.com
```

对远程主机使用 HTTPS，以保护工具参数和结果。仅可在回环地址上 或通过安全隧道使用 HTTP。该 URL 接受主机名和可选端口。 请省略凭据、路径、查询字符串和片段。使用 `--listen` 配置从客户端到 app-server 的独立连接。

协议

与 MCP 类似，`codex app-server` 支持使用 JSON-RPC 2.0 消息进行双向通信（在线路上传输时省略 `"jsonrpc":"2.0"` 头）。

支持的传输方式：

- `stdio`（`--listen stdio://`，默认）：换行分隔的 JSON（JSONL）。
- `websocket`（`--listen ws://IP:PORT`，实验性且不受支持）：每个 WebSocket 文本帧承载一条 JSON-RPC 消息。
- Unix socket（`--listen unix://` 或 `--listen unix://PATH`）：通过 Codex 默认的 app-server 控制 socket 或自定义 Unix socket 路径建立 WebSocket 连接，采用标准 HTTP Upgrade 握手。
- `off`（`--listen off`）：不暴露本地传输。

当你以 `--listen ws://IP:PORT` 运行时，同一监听器还会提供基本的 HTTP 健康探测：

- `GET /readyz` 在监听器开始接受新连接后返回 `200 OK`。
- 当请求不包含 `Origin` 头时，`GET /healthz` 返回 `200 OK`。
- 带有 `Origin` 头的请求会被以 `403 Forbidden` 拒绝。

WebSocket 传输是实验性的且不受支持。诸如 `ws://127.0.0.1:PORT` 这样的本地监听器适用于 localhost 和 SSH 端口转发场景。在 rollout 期间，非回环 WebSocket 监听器目前默认允许未认证的连接，因此 在远程暴露此类监听器之前，请先配置 WebSocket 认证。

支持的 WebSocket 认证标志：

- `--ws-auth capability-token --ws-token-file /absolute/path`
- `--ws-auth capability-token --ws-token-sha256 HEX`
- `--ws-auth signed-bearer-token --ws-shared-secret-file /absolute/path`

对于签名的 bearer token，你还可以设置 `--ws-issuer`、`--ws-audience` 和 `--ws-max-clock-skew-seconds`。客户端在 WebSocket 握手期间以 `Authorization: Bearer <token>` 形式出示凭据，app-server 会在 JSON-RPC `initialize` 之前强制执行认证。

优先使用 `--ws-token-file`，而不是在命令行上传递原始 bearer token。仅当 客户端把原始高熵 token 保存在单独的本地机密存储中时，才使用 `--ws-token-sha256`；哈希仅作校验之用，客户端仍然 需要原始 token。

在 WebSocket 模式下，app-server 使用有界队列。当请求入口队列已满时， 服务器会以 JSON-RPC 错误码 `-32001` 和消息 `"Server overloaded; retry later."` 拒绝新请求。客户端应采用指数递增 延迟加抖动的方式进行重试。

消息结构（schema）

请求包含 `method`、`params` 和 `id`。

本文示例使用 GPT-6.1 Sol，该模型要求已登录的账户或工作区 具有相应的访问权限。如有需要，可使用 `model/list` 选择一个可用的模型。

```
{ "method": "thread/start", "id": 10, "params": { "model": "gpt-6.1-sol" } }
```

响应会回显 `id`，并附带 `result` 或 `error`：

```
{ "id": 10, "result": { "thread": { "id": "thr_123" } } }
```

```
{ "id": 10, "error": { "code": 123, "message": "Something went wrong" } }
```

通知省略 `id`，只使用 `method` 和 `params`：

```
{ "method": "turn/started", "params": { "turn": { "id": "turn_456" } } }
```

你可以从 CLI 生成 TypeScript schema 或 JSON Schema 包。每份输出都特定于你所运行的 Codex 版本，因此生成的工件与该版本完全一致：

```
codex app-server generate-ts --out ./schemas
codex app-server generate-json-schema --out ./schemas
```

快速上手

1. 使用 `codex app-server`（默认 stdio 传输）、`codex app-server --listen ws://127.0.0.1:4500`（TCP WebSocket）或 `codex app-server --listen unix://`（默认 Unix socket）启动服务器。
2. 通过所选传输方式连接客户端，然后发送 `initialize`，随后发送 `initialized` 通知。
3. 启动一个线程和一个轮次（turn），然后持续从活动传输流中读取通知。

示例（Node.js / TypeScript）：

```
import { spawn } from "node:child_process";
import readline from "node:readline";
const proc = spawn("codex", ["app-server"], {
  stdio: ["pipe", "pipe", "inherit"],
});
const rl = readline.createInterface({ input: proc.stdout });
const send = (message: unknown) => {
  proc.stdin.write(`${JSON.stringify(message)}\n`);
};
let threadId: string | null = null;
rl.on("line", (line) => {
  const msg = JSON.parse(line) as any;
  console.log("server:", msg);
```

```
  if (msg.id === 0 && msg.result) {
    send({ method: "initialized", params: {} });
    send({
      method: "thread/start",
      id: 1,
      params: { model: "gpt-6.1-sol" },
    });
  }
  if (msg.method === "turn/completed") {
    proc.stdin.end();
    rl.close();
  }
  if (msg.error) {
    console.error(msg.error);
    process.exitCode = 1;
    proc.stdin.end();
    rl.close();
  }
  if (msg.id === 1 && msg.result?.thread?.id && !threadId) {
    threadId = msg.result.thread.id;
    send({
      method: "turn/start",
      id: 2,
      params: {
        threadId,
        input: [{ type: "text", text: "Summarize this repo." }],
      },
    });
  }
});
send({
  method: "initialize",
  id: 0,
  params: {
    clientInfo: {
      name: "my_product",
      title: "My Product",
      version: "0.1.0",
    },
  },
});
proc.on("error", (error) => {
  console.error(error);
  process.exitCode = 1;
  rl.close();
});
```

核心原语

- **线程（Thread）**：用户与 Codex 智能体之间的一段对话。线程包含轮次。
- **轮次（Turn）**：一次用户请求以及随后的智能体工作。轮次包含条目（item），并流式传输增量更新。
- **条目（Item）**：输入或输出的一个单元（用户消息、智能体消息、命令运行、文件变更、工具调用等）。

使用线程 API 创建、列出或归档对话。通过轮次 API 驱动对话，并通过轮次通知流式传输进度。

生命周期概览

- **每连接初始化一次**：打开传输连接后，立即发送携带你客户端元数据的 `initialize` 请求，然后发出 `initialized`。在此握手之前，服务器会拒绝该连接上的任何请求。
- **启动（或恢复）线程**：调用 `thread/start` 开始新对话，调用 `thread/resume` 继续已有对话，或调用 `thread/fork` 将历史分支到一个新的线程 id。
- **开始一个轮次**：以目标 `threadId` 和用户输入调用 `turn/start`。可选字段可覆盖模型、工作目录和沙箱策略。
- **引导活动轮次**：调用 `turn/steer`，在不创建新轮次的情况下向当前进行中的轮次追加用户输入。
- **流式传输事件**：`turn/start` 之后，持续读取 stdout 上的通知：`thread/archived`、`thread/unarchived`、`item/started`、`item/completed`、`item/agentMessage/delta`、工具进度以及其他更新。
- **结束轮次**：当模型完成工作或经过 `turn/interrupt` 取消后，服务器会发出携带最终状态的 `turn/completed`。

初始化

客户端必须在每条传输连接上、于调用该连接上的任何其他方法之前，恰好发送一次 `initialize` 请求，然后以 `initialized` 通知进行确认。在初始化之前发送的请求会收到 `Not initialized` 错误，而在同一连接上重复调用 `initialize` 会返回 `Already initialized`。

服务器会返回它将呈现给上游服务的用户代理字符串，以及描述运行时目标的 `platformFamily` 和 `platformOs` 值。请设置 `clientInfo` 以标识你的集成。

`initialize.params.capabilities` 还支持以下客户端能力：

- `optOutNotificationMethods` – 要在此连接上抑制的确切通知方法名。匹配为 精确匹配（不支持通配符或前缀）；未知名称会被 接受并忽略。
- `requestAttestation` – 选择加入服务器发起的 `attestation/generate` 请求。提供上游证明的桌面主机会以一个 不透明的 `{ "token": "..." }` 值响应。
- `mcpServerOpenaiFormElicitation` – 允许下游 MCP 服务器发送 `mcpServer/elicitation/request` 的 OpenAI 扩展表单变体。

重要：请使用 `clientInfo.name` 为 OpenAI Compliance Logs Platform 标识你的客户端。如果你正在开发面向企业使用的新 Codex 集成，请联系 OpenAI 将其加入已知客户端列表。更多背景信息请参见 Codex 日志参考。

示例（来自 Codex VS Code 扩展）：

```
{
  "method": "initialize",
  "id": 0,
  "params": {
    "clientInfo": {
      "name": "codex_vscode",
      "title": "Codex VS Code Extension",
```

```
      "version": "0.1.0"
    }
  }
}
```

带通知退出的示例：

```
{
  "method": "initialize",
  "id": 1,
  "params": {
    "clientInfo": {
      "name": "my_client",
      "title": "My Client",
      "version": "0.1.0"
    },
    "capabilities": {
      "experimentalApi": true,
      "optOutNotificationMethods": ["thread/started", "item/agentMessage/delta"]
    }
  }
}
```

实验性 API 选择加入

一些 app-server 方法和字段被有意地置于 `experimentalApi` 能力门控之后。

- 省略 `capabilities`（或将 `experimentalApi` 设为 `false`）以停留在稳定的 API 表面上，此时服务器会拒绝实验性方法/字段。
- 将 `capabilities.experimentalApi` 设为 `true` 以启用实验性方法和字段。

```
{
  "method": "initialize",
  "id": 1,
  "params": {
    "clientInfo": {
      "name": "my_client",
      "title": "My Client",
      "version": "0.1.0"
    },
    "capabilities": {
      "experimentalApi": true
    }
  }
}
```

如果客户端在未选择加入的情况下发送实验性方法或字段，app-server 会以下列消息拒绝：

`<descriptor> requires experimentalApi capability`

API 概览

- `thread/start` – 创建一个新线程；发出 `thread/started`，并自动为你订阅该线程的轮次/条目事件。
- `thread/resume` – 按 id 重新打开一个已有线程，使后续的 `turn/start` 调用追加到该线程。
- `thread/fork` – 通过复制已存储的历史，将一个线程分叉到一个新的线程 id。传入 `lastTurnId` 可只复制到该轮次为止的历史并省略之后的轮次，或传入 `ephemeral: true` 创建一个内存中的分叉。为新线程发出 `thread/started`；返回的线程在可用时包含 `forkedFromId`。
- `thread/read` – 按 id 读取已存储的线程元数据，而不恢复它。使用 `thread/turns/list` 和 `thread/items/list` 加载历史。返回的 `thread` 对象包含运行时 `status`。
- `thread/list` – 分页浏览已存储的线程及其运行时状态。参见分页与过滤器。
- `thread/turns/list` – 分页浏览已存储线程的轮次历史，而不恢复它。`itemsView` 控制轮次条目是被省略、摘要显示还是完整加载。
- `thread/items/list` – 分页浏览已持久化的线程条目，可选地限定于单个 `turnId`。活动线程存储必须支持条目分页。
- `thread/loaded/list` – 列出当前加载到内存中的线程 id。
- `thread/name/set` – 为已加载的线程或已持久化的 rollout 设置或更新线程的面向用户名称；发出 `thread/name/updated`。
- `thread/goal/set` – 为线程设置目标；发出 `thread/goal/updated`。
- `thread/goal/get` – 读取线程的当前目标。
- `thread/goal/clear` – 清除线程的目标；发出 `thread/goal/cleared`。
- `thread/metadata/update` – 更新为线程存储的 Git 元数据。
- `thread/archive` – 将线程的日志文件移入已归档目录，并尝试归档尚未归档的派生后代线程日志；成功时返回 `{}`，并为每个已归档的线程发出 `thread/archived`。
- `thread/delete` – 永久删除一个已持久化的活动或已归档线程及其所有派生后代线程；成功时返回 `{}`，并为每个已删除的线程发出 `thread/deleted`。
- `thread/unsubscribe` – 让此连接退订线程的轮次/条目事件。如果这是最后一个订阅者，服务器会在无订阅者不活动宽限期之后卸载该线程并发出 `thread/closed`。
- `thread/unarchive` – 将已归档线程的 rollout 恢复到活动会话目录；返回恢复后的 `thread` 并发出 `thread/unarchived`。
- `thread/status/changed` – 当已加载线程的运行时 `status` 发生变化时发出的通知。
- `thread/compact/start` – 为线程触发对话历史压缩（compaction）；立即返回 `{}`，进度通过 `turn/*` 和 `item/*` 通知流式传输。
- `thread/shellCommand` – 针对线程运行一条由用户发起的 shell 命令。该命令在沙箱之外运行、拥有完全访问权限，并且不继承线程的沙箱策略。
- `thread/backgroundTerminals/clean` – 停止一个线程的所有运行中后台终端（实验性；需要 `capabilities.experimentalApi`）。
- `thread/backgroundTerminals/list` – 列出一个已加载线程的运行中后台终端（实验性；需要 `capabilities.experimentalApi`）。

- `thread/backgroundTerminals/terminate` – 按 app-server 的 `processId` 终止一个运行中的后台终端（实验性；需要 `capabilities.experimentalApi`）。
- `thread/revert` – 从持久化分页线程的历史中移除指定轮次及其之后的所有轮次；返回更新后的元数据和历史游标，然后发出 `thread/reverted`。
- `turn/start` – 向线程添加用户输入或独立的工具输出，并开始 Codex 生成；响应包含初始 `turn` 并流式传输事件。对于 `collaborationMode`，`settings.developer_instructions: null` 表示“对所选模式使用内置指令”。
- `thread/inject_items` – 向已加载线程的模型可见历史追加原始 Responses API 条目，而不启动用户轮次。
- `turn/steer` – 向线程当前进行中的轮次追加用户输入；返回被接受的 `turnId`。
- `turn/interrupt` – 请求取消一个进行中的轮次；成功时返回 `{}`，轮次以 `status: "interrupted"` 结束。
- `review/start` – 为线程启动 Codex 审查器；发出 `enteredReviewMode` 和 `exitedReviewMode` 条目。
- `command/exec` – 在服务器沙箱下运行单条命令，而不启动线程/轮次。
- `command/exec/write` – 向运行中的 `command/exec` 会话写入 `stdin` 字节，或关闭 `stdin`。
- `command/exec/resize` – 调整运行中由 PTY 支撑的 `command/exec` 会话的大小。
- `command/exec/terminate` – 停止一个运行中的 `command/exec` 会话。
- `command/exec/outputDelta`（通知）– 针对流式传输的 `command/exec` 会话发出的 base64 编码 stdout/stderr 数据块。
- `process/spawn` – 在 Codex 沙箱之外启动一个显式进程会话（实验性；需要 `capabilities.experimentalApi`）。
- `process/writeStdin` – 向运行中的 `process/spawn` 会话写入 stdin 字节，或关闭 stdin（实验性）。
- `process/resizePty` – 调整运行中由 PTY 支撑的进程会话的大小（实验性）。
- `process/kill` – 终止一个运行中的进程会话（实验性）。
- `process/outputDelta` 和 `process/exited`（通知）– 针对流式传输的进程输出和进程退出状态发出（实验性）。
- `model/list` – 列出可用模型（设置 `includeHidden: true` 可包含 `hidden: true` 的条目），附带推理力度选项、可选的 `upgrade` 以及 `inputModalities`。
- `modelProvider/capabilities/read` – 读取模型/提供方组合的提供方能力边界。
- `experimentalFeature/list` – 列出功能标志，附带生命周期阶段元数据和游标分页。
- `experimentalFeature/enablement/set` – 为受支持的功能键（例如 `apps` 和 `plugins`）修补内存中的运行时设置。
- `environment/info` – 实验性；连接到一个已配置的执行环境，并返回其 shell 和默认工作目录。
- `permissionProfile/list` – 列出 beta 权限配置文件（permission profile）以及有效需求是否允许使用它们，支持游标分页。
- `collaborationMode/list` – 列出协作模式预设（实验性，无分页）。

- `skills/list` – 列出一个或多个 `cwd` 值对应的技能（支持 `forceReload` 和可选的 `perCwdExtraUserRoots`）。
- `skills/extraRoots/set` – 替换用于发现独立技能的进程级额外根目录，而不将其持久化。
- `skills/changed`（通知）– 当受监视的本地技能文件发生变化时发出。
- `hooks/list` – 列出一个或多个 `cwd` 值所发现的生命周期钩子。
- `marketplace/add` – 添加一个远程插件市场，并将其持久化到用户的 marketplace 配置中。
- `marketplace/remove` – 移除一个已配置的市场，并在存在时移除其已安装的市场根目录。
- `marketplace/upgrade` – 刷新一个已配置的 Git 市场，或在省略市场名称时刷新所有已配置的 Git 市场。
- `plugin/list` – 开发中；列出已发现的插件市场和插件状态，包括安装/认证策略元数据、市场加载错误、精选插件 id，以及本地、Git、包注册表或远程插件来源元数据。摘要可以包含远程 `version`、本地 `localVersion`、结构化的浅色/深色图标，以及 `installPolicySource`（对于当前的远程条目，它可以是 `null`、`WORKSPACE_SETTING` 或 `IMPLICIT_CANONICAL_APP`）。请勿在生产客户端中调用此方法。
- `plugin/read` – 开发中；按市场路径，或按远程市场名称加插件名称读取一个插件，包括捆绑的技能、应用、MCP 服务器名称，以及远程目录提供时的远程插件 `shareUrl`。请勿在生产客户端中调用此方法。
- `plugin/install` – 开发中；从市场路径或远程市场名称安装插件。请勿在生产客户端中调用此方法。
- `plugin/uninstall` – 开发中；卸载一个已安装的插件。请勿在生产客户端中调用此方法。
- `plugin/skill/read` – 按远程市场、插件 id 和技能名按需读取远程插件技能的 Markdown。
- `app/installed` – 读取已安装应用的运行时状态，包括每个应用的有效启用状态和可调用状态。
- `app/list` – 分页列出可用应用（连接器），附带可访问性/启用元数据。
- `app/read` – 获取特定应用 id 的元数据，以及可选的仅用于展示的工具摘要。
- `skills/config/write` – 按路径启用或禁用技能。
- `mcpServer/oauth/login` – 为一个已配置的 MCP 服务器启动 OAuth 登录。打开返回的 HTTP 或 HTTPS `authorizationUrl`，并将 `loginId` 与 `mcpServer/oauthLogin/completed` 通知相匹配。
- `tool/requestUserInput` – 针对一次工具调用向用户提出 1–3 个简短问题（实验性）；问题可以设置 `isOther` 以提供自由填写选项。
- `mcpServer/elicitation/request`（服务器请求）– 请求客户端提供结构化表单输入，或确认由 MCP 服务器发起的 URL 流程。
- `item/permissions/requestApproval`（服务器请求）– 请求客户端授予内置 `request_permissions` 工具所请求的网络或文件系统权限的一个子集。
- `config/mcpServer/reload` – 从磁盘重新加载 MCP 服务器配置，并为已加载的线程排队一次刷新。
- `mcpServerStatus/list` – 列出 MCP 服务器、工具、资源和认证状态（游标 + limit 分页）。使用 `detail: "full"` 获取完整数据，或使用 `detail: "toolsAndAuthOnly"` 省略资源。两种细节模式都包含服务器通告的 `serverCapabilities`，或在成功初始化之前为 `null`。传入 `serverName` 可发现单个服务器。传入 `threadId` 时，该请求会复用该线程的 MCP 连接。
- `mcpServer/resource/read` – 通过一个已初始化的 MCP 服务器读取单个 MCP 资源。

- `mcpServer/tool/call` – 在线程已配置的 MCP 服务器上调用工具。
- `mcpServer/startupStatus/updated`（通知）– 当已配置 MCP 服务器针对某个已加载线程的启动状态发生变化时发出。
- `windowsSandbox/readiness` – 检查 Windows 沙箱是否就绪。如果返回 `updateRequired`，请调用 `windowsSandbox/setupStart` 并等待 `windowsSandbox/setupCompleted`。
- `feedback/upload` – 提交反馈报告（分类 + 可选的原因/日志 + 会话 id，外加可选的 `extraLogFiles` 附件）。
- `config/read` – 在解析配置分层之后，获取磁盘上的有效配置。
- `externalAgentConfig/detect` – 检测可通过 `includeHome` 和可选 `cwds` 迁移的外部智能体工件；每个检测到的条目都包含 `cwd`（home 为 `null`）。
- `externalAgentConfig/import` – 通过传入带 `cwd`（home 为 `null`）的显式 `migrationItems`，应用所选的外部智能体迁移条目。受支持的条目类型包括配置、技能、`AGENTS.md`、插件、MCP 服务器配置、子智能体（subagent）、钩子、命令和会话；非空导入会随着工作完成发出 `externalAgentConfig/import/progress` 和 `externalAgentConfig/import/completed`。插件和会话导入可能异步完成。
- `config/value/write` – 向磁盘上用户的 `config.toml` 写入单个配置键/值。
- `config/batchWrite` – 以原子方式向磁盘上用户的 `config.toml` 应用配置编辑。
- `configRequirements/read` – 从 `requirements.toml` 和/或 MDM 获取需求，包括允许列表和固定的 `featureRequirements`。启用了 `experimentalApi` 的客户端还可以读取托管网络需求。
- `fs/readFile`、`fs/writeFile`、`fs/createDirectory`、`fs/getMetadata`、`fs/readDirectory`、`fs/remove`、`fs/copy`、`fs/watch`、`fs/unwatch` 和 `fs/changed`（通知）– 通过 app-server v2 文件系统 API 对绝对文件系统路径进行操作。

插件摘要包含一个 `source` 联合类型。本地插件返回 `{ "type": "local", "path": ... }`，Git 支撑的市场条目返回 `{ "type": "git", "url": ..., "path": ..., "refName": ..., "sha": ... }`， 包注册表条目返回 `{ "type": "npm", "package": ..., "version": ..., "registry": ... }`， 而远程目录条目返回 `{ "type": "remote" }`。对于仅远程的目录 条目，`PluginMarketplaceEntry.path` 可以为 `null`；在读取或 安装此类插件时，请传入 `remoteMarketplaceName` 而不是 `marketplacePath`。

模型

列出模型（`model/list`）

调用 `model/list`，用可用的模型和推理力度（reasoning effort）填充模型选择器。

下面的响应展示了其结构。可用模型、推理力度和默认值 取决于客户端与账户；请使用返回的值， 而不要把此示例硬编码。

```
{ "method": "model/list", "id": 6, "params": { "limit": 20, "includeHidden": false } }
{ "id": 6, "result": {
  "data": [{
    "id": "gpt-6.1-sol",
    "model": "gpt-6.1-sol",
```

```
    "displayName": "GPT-6.1 Sol",
    "hidden": false,
    "defaultReasoningEffort": "medium",
    "supportedReasoningEfforts": [{
      "reasoningEffort": "medium",
      "description": "Balances speed and reasoning depth for everyday tasks"
    }],
    "inputModalities": ["text", "image"],
    "supportsPersonality": false,
    "isDefault": true
  }],
  "nextCursor": null
} }
```

每个模型条目可以包含：

- `supportedReasoningEfforts` – 该模型支持的推理力度选项。
- `defaultReasoningEffort` – 建议客户端使用的默认力度。
- `upgrade` – 可选的推荐升级模型 id，用于客户端中的迁移提示。
- `upgradeInfo` – 可选的升级元数据，用于客户端中的迁移提示。
- `hidden` – 该模型是否从默认选择器列表中隐藏。
- `inputModalities` – 该模型支持的输入类型（例如 `text`、`image`）。
- `supportsPersonality` – 已弃用的兼容字段；始终为 `false`。模型不再支持个性选择。
- `isDefault` – 该模型是否为推荐的默认模型。

如果启动时的提供方不再满足托管需求，`model/list` 会返回 `-32600` 并要求客户端重启 Codex。

默认情况下，`model/list` 仅返回选择器可见的模型。如果你需要完整列表并希望在客户端侧使用 `hidden` 进行过滤，请设置 `includeHidden: true`。

当 `inputModalities` 缺失（较旧的模型目录）时，出于向后兼容，请将其视为 `["text", "image"]`。

列出实验性功能（`experimentalFeature/list`）

使用此端点可以发现带有元数据和生命周期阶段的功能标志：

```
{ "method": "experimentalFeature/list", "id": 7, "params": { "limit": 20 } }
{ "id": 7, "result": {
  "data": [{
    "name": "unified_exec",
    "stage": "beta",
    "displayName": "Unified exec",
    "description": "Use the unified PTY-backed execution tool.",
    "announcement": "Beta rollout for improved command execution reliability.",
    "enabled": false,
    "defaultEnabled": false
  }],
  "nextCursor": null
} }
```

`stage` 可以是 `beta`、`underDevelopment`、`stable`、`deprecated` 或 `removed`。对于非 beta 标志，`displayName`、`description` 和 `announcement` 可能为 `null`。

检查执行环境（实验性）

在开始在那里工作之前，使用 `environment/info` 检查一个已配置的远程环境。 该方法要求 `capabilities.experimentalApi = true`。

```
{ "method": "environment/info", "id": 8, "params": { "environmentId": "devbox" } }
{ "id": 8, "result": {
  "shell": { "name": "zsh", "path": "/bin/zsh" },
  "cwd": "file:///workspace/project"
} }
```

`cwd` 可以为 `null`。存在时，它是一个使用该环境 原生路径语法的规范 `file:` URI。未知的环境 ID 以及连接 或协议失败会返回请求错误。

线程

- `thread/read` 读取已存储的线程元数据，而不订阅它。
- `thread/turns/list` 分页浏览已存储线程的轮次历史，而不 恢复它。使用 `itemsView` 选择轮次条目是被省略、摘要 显示还是完整加载。
- `thread/items/list` 分页浏览已持久化的线程条目，可选地限定于单个轮次。
- `thread/list` 分页浏览已存储的线程。使用 `sectionId` 列出某个分区中的线程，或使用 `excludedThreadIds` 省略屏幕上已显示的线程。
- `thread/loaded/list` 返回当前位于内存中的线程 ID。
- `thread/archive` 将线程已持久化的 JSONL 日志移入已归档目录，并尝试归档尚未归档的派生后代线程日志。
- `thread/delete` 永久删除一个已持久化的活动或已归档线程及其派生后代线程。
- `thread/metadata/update` 更新为线程存储的 `gitInfo`。
- `thread/unsubscribe` 让当前连接退订一个已加载的线程，并可在不活动宽限期后触发 `thread/closed`。
- `thread/unarchive` 将已归档线程的 rollout 恢复到活动会话目录。
- `thread/compact/start` 触发压缩（compaction）并立即返回 `{}`。
- `thread/revert` 从持久化分页线程的历史中移除指定轮次及其之后的所有轮次。它不会更改本地文件。
- `thread/inject_items` 向已加载线程的模型可见历史追加原始 Responses API 条目，而不启动用户轮次。

启动或恢复线程

当你需要一段新的 Codex 对话时，请启动一个全新的线程。

```
{ "method": "thread/start", "id": 10, "params": {
  "model": "gpt-6.1-sol",
  "cwd": "/Users/me/project",
```

```
  "approvalPolicy": "never",
  "sandbox": "workspace-write",
  "serviceName": "my_app_server_client"
} }
{ "id": 10, "result": {
  "thread": {
    "id": "thr_123",
    "sessionId": "thr_123",
    "preview": "",
    "ephemeral": false,
    "modelProvider": "openai",
    "createdAt": 1730910000
  }
} }
{ "method": "thread/started", "params": { "thread": { "id": "thr_123" } } }
```

`serviceName` 是可选的。当你希望 app-server 用你集成的服务名标记线程级指标时，请设置它。

`thread/start`、`thread/resume` 和 `thread/fork` 会返回 `instructionSources`，即已加载的指令文件路径数组。每个路径都使用 其来源环境的原生绝对路径语法，包括远程 环境。

新的持久化本地线程默认使用分页历史。请使用 `thread/turns/list` 和 `thread/items/list` 加载历史。在 `thread/start` 上 显式地将 `historyMode` 设为 `"legacy"` 或 `"paginated"` 属于 实验性特性，需要 `capabilities.experimentalApi = true`。

选择加入 `capabilities.experimentalApi` 的 beta 客户端可以在 `permissions` 中传入具名的权限配置文件 id，以替代旧版 `sandbox` 字段。请不要同时发送 `permissions` 和 `sandbox`。将 `permissionProfile/list` 与项目 `cwd` 一起使用，可发现可用的 配置文件，以及托管需求是否允许使用各个配置文件。

`thread.sessionId` 标识当前的活动会话树根。根线程使用 自身的线程 id 作为会话 id；分叉线程保留其来源 根线程的会话 id。客户端应从 `thread.sessionId` 读取会话 id，而不是从线程 id 推导。

要继续一个已存储的会话，请使用你先前记录的 `thread.id` 调用 `thread/resume`。传入 `excludeTurns: true` 可返回线程元数据和活动会话状态，然后使用分页方法加载历史：

```
{ "method": "thread/resume", "id": 11, "params": {
  "threadId": "thr_123",
  "excludeTurns": true
} }
{ "id": 11, "result": { "thread": { "id": "thr_123", "name": "Bug bash notes", "ephemeral": false } } }
```

对于分页线程，`thread/resume` 和 `thread/fork` 的完整历史响应已弃用，并会发出 `deprecationNotice`。这两个方法 无需实验性 API 选择加入即可接受 `excludeTurns: true`。`thread/resume` 的响应 还可以返回 `turnsBackwardsCursor` 和 `itemsBackwardsCursor`。请将它们分别 传给对应的分页方法并附带 `sortDirection: "desc"`；第一页 包含游标的锚点。

恢复线程本身不会更新 `thread.updatedAt`（或 rollout 文件的修改时间）。当你启动一个轮次时，时间戳才会更新。

如果你在配置中将某个已启用的 MCP 服务器标记为 `required`，而该服务器初始化失败，那么 `thread/start` 和 `thread/resume` 会直接失败，而不是在没有它的情况下继续。

`thread/start` 上的 `dynamicTools` 是一个实验性字段（需要 `capabilities.experimentalApi = true`）。Codex 会将这些动态工具持久化到线程的 rollout 元数据中，并在你不提供新动态工具的情况下于 `thread/resume` 时恢复它们。

如果配置的模型提供方与线程保存的提供方不同， `thread/resume` 和新输入会返回 `-32600`，且 `data.type` 设为 `modelProviderMismatch`。要显示线程的历史，请使用 `thread/turns/list` 分页浏览其轮次。切换回已保存的提供方以继续 该线程，或调用 `thread/fork` 以当前提供方从其历史 启动一个新线程。

如果你恢复时使用的模型与 rollout 中记录的模型不同，Codex 会发出警告，并在下一个轮次应用一次性的模型切换指令。

管理线程（thread）目标

使用 `thread/goal/set`、`thread/goal/get` 和 `thread/goal/clear` 来管理与 TUI 中 `/goal` 所呈现的同一份持久化目标状态。

```
{ "method": "thread/goal/set", "id": 13, "params": {
  "threadId": "thr_123",
  "objective": "Finish the migration and keep tests green",
  "status": "active",
  "tokenBudget": 40000
} }
{ "id": 13, "result": { "goal": {
  "threadId": "thr_123",
  "objective": "Finish the migration and keep tests green",
  "status": "active",
  "tokenBudget": 40000,
  "tokensUsed": 0,
  "timeUsedSeconds": 0
} } }
{ "method": "thread/goal/updated", "params": {
  "threadId": "thr_123",
  "goal": {
    "threadId": "thr_123",
    "objective": "Finish the migration and keep tests green",
    "status": "active",
    "tokenBudget": 40000,
    "tokensUsed": 0,
    "timeUsedSeconds": 0
  }
} }
```

目标的 objective 必须非空且最多 4,000 个字符。提供新的 objective 会替换目标并重置用量统计。提供当前处于非终止状态的 objective，或省略 `objective`，会在保留用量历史的同时更新状态或 token 预算。

要从已存储的会话（session）创建分叉（fork），请携带 `thread.id` 调用 `thread/fork`。这会创建一个新的线程 id，并为其发出一条 `thread/started` 通知。传入 `lastTurnId` 可复制截至该轮次（turn）（含）的历史，并排除之后的轮次：

```
{ "method": "thread/fork", "id": 12, "params": { "threadId": "thr_123", "lastTurnId": "turn_456",
"excludeTurns": true } }
{ "id": 12, "result": { "thread": { "id": "thr_456", "sessionId": "thr_123", "forkedFromId": "thr_123"
} } }
{ "method": "thread/started", "params": { "thread": { "id": "thr_456" } } }
```

App-server 会拒绝指向进行中轮次的 `lastTurnId`。如果源线程正处于轮次进行中而你省略了该字段，分叉会记录一个中断标记，而不是保留一个未标记的不完整轮次。

传入 `ephemeral: true` 可创建仅存在于内存中的分叉，且不会将其添加到已存储的线程列表中：

```
{
  "method": "thread/fork",
  "id": 13,
  "params": {
    "threadId": "thr_123",
    "ephemeral": true,
    "excludeTurns": true
  }
}
{
  "id": 13,
  "result": {
    "thread": {
      "id": "thr_789",
      "sessionId": "thr_789",
      "forkedFromId": "thr_123",
      "ephemeral": true
    }
  }
}
```

对分页线程创建 ephemeral 分叉时要求 `excludeTurns: true`。

当已设置面向用户的线程标题时，app-server 会在 `thread/list`、`thread/read`、`thread/resume` 和 `thread/unarchive` 的响应中填充 `thread.name`。在稍后设置标题之前，`thread/start` 和 `thread/fork` 可能会省略 `name`（或返回 `null`）。

读取已存储的线程（不恢复）

当你想获取已存储的线程数据、但不想恢复线程或订阅其事件时，请使用 `thread/read`。

- `includeTurns` – 为 `false` 或省略时，仅返回线程摘要。将其设为 `true` 会加载完整历史，这对分页线程而言已被弃用，并且会发出 `deprecationNotice`。请改用分页方法。
- 返回的 `thread` 对象包含运行时 `status`（`notLoaded`、`idle`、`systemError`，或带 `activeFlags` 的 `active`）。

```
{ "method": "thread/read", "id": 19, "params": { "threadId": "thr_123" } }
{ "id": 19, "result": { "thread": { "id": "thr_123", "name": "Bug bash notes", "ephemeral": false,
"status": { "type": "notLoaded" }, "turns": [] } } }
```

与 `thread/resume` 不同，`thread/read` 不会将线程加载到内存中，也不会发出 `thread/started`。

列出线程轮次

使用 `thread/turns/list` 可在不恢复已存储线程的情况下对其轮次历史分页。结果默认按从新到旧排序，客户端因此可以通过 `nextCursor` 获取更早的轮次。响应中还包含 `backwardsCursor`；将它作为 `cursor` 并配合相反的 `sortDirection` 传入，即可反向分页。此时会再次包含锚点轮次，以便你捕获它的更新。

`itemsView` 控制响应包含多少轮次条目（item）数据：

- `notLoaded` 省略条目。
- `summary` 返回摘要式条目数据，是省略该字段时的默认值。
- `full` 返回完整条目数据。

```
{ "method": "thread/turns/list", "id": 20, "params": {
  "threadId": "thr_123",
  "limit": 50,
  "sortDirection": "desc",
  "itemsView": "summary"
} }
{ "id": 20, "result": {
  "data": [],
  "nextCursor": "older-turns-cursor-or-null",
  "backwardsCursor": "newer-turns-cursor-or-null"
} }
```

列出线程条目

`thread/items/list` 在不恢复线程的情况下对已持久化的条目分页。传入 `turnId` 可将结果限定在单个轮次内，省略它则在整个线程范围内对条目分页。结果默认按从旧到新排序（`sortDirection: "asc"`）。当前的线程存储必须支持条目分页；否则，服务器会返回“不支持的方法”错误。

传入上一次响应返回的不透明 `nextCursor` 以继续分页。要从某个已知条目之后开始，请传入一个条目锚点和一个非空的 `turnId`：

```
{ "method": "thread/items/list", "id": 21, "params": {
  "threadId": "thr_123",
  "turnId": "turn_456",
  "cursor": { "type": "item", "itemId": "item_123" },
  "sortDirection": "asc",
  "limit": 50
} }
{ "id": 21, "result": {
  "data": [],
  "nextCursor": null,
  "backwardsCursor": null
} }
```

条目锚点会排除锚点自身：升序返回更新的条目，降序返回更早的条目。该条目必须属于所请求轮次的可见历史。缺少 `turnId`、条目 ID 为空，或条目未知、超出范围，都会返回 invalid params（`-32602`）。使用返回的字符串 `nextCursor` 继续。每个结果条目包含 `turnId`、`item`，以及可选的已记录开始时间和完成时间戳。

列出线程（支持分页与过滤）

`thread/list` 让你能够渲染历史 UI。结果默认按 `createdAt` 从新到旧排序。过滤器在分页之前应用。可传入以下任意组合：

- `cursor` – 上一次响应返回的不透明字符串；获取第一页时省略。
- `limit` – 未设置时，服务器会使用合理的默认页大小。
- `sortKey` – `created_at`（默认）、`updated_at`、`recency_at` 或 `section_position`。要使用 `section_position`，需提供非 null 的 `sectionId`。
- `sortDirection` – `asc` 或 `desc`。如果省略 `sortDirection`，`section_position` 按升序排序，基于时间的键按降序排序。
- `sectionId` – 传入某个 section 的 ID 以列出该 section 中的线程，传 `null` 表示列出未分配到任何 section 的线程。省略该字段则同时包含所有 section 中的线程和未分配的线程。
- `excludedThreadIds` – 最多排除 100 个线程 ID。重复的 ID 也计入这 100 个 ID 的上限。服务器会在应用页面限制之前排除匹配的线程。随每个 cursor 请求发送相同的排除列表。
- `modelProviders` – 将结果限制为特定的提供方（provider）。传入 `[]` 表示包含所有提供方。如果省略该字段或传入 `null`，服务器会使用已配置的提供方，除非你按线程关系进行过滤——这种情况下会包含所有提供方。
- `sourceKinds` – 将结果限制为特定的线程来源。省略或传入 `[]` 时，服务器默认仅包含交互式来源：`cli` 和 `vscode`。
- `archived` – 为 `true` 时，仅列出已归档线程。为 `false` 或省略时，列出未归档线程（默认）。
- `cwd` – 将结果限制为会话当前工作目录与该路径（或数组中的某个路径）完全匹配的线程。相对路径基于 app-server 进程的工作目录进行解析。
- `useStateDbOnly` – 为 `true` 时，返回状态数据库的结果，不扫描 JSONL 线程日志来修复元数据。省略该字段或传入 `false` 则采用默认的扫描并修复行为。
- `searchTerm` – 将结果限制为所提取标题包含该区分大小写的文本片段的线程。
- `parentThreadId` – 将结果限制为给定父线程的直接子线程。此过滤器是实验性的，要求 `capabilities.experimentalApi = true`。
- `ancestorThreadId` – 将结果限制为给定线程在任意深度的派生后代。此过滤器是实验性的，要求 `capabilities.experimentalApi = true`；不要将其与 `parentThreadId` 组合使用。

`sourceKinds` 接受以下取值：

- `cli`
- `vscode`
- `exec`
- `appServer`
- `subAgent`
- `subAgentReview`
- `subAgentCompact`
- `subAgentThreadSpawn`

- `subAgentOther`
- `unknown`

示例：

```
{ "method": "thread/list", "id": 20, "params": {
  "cursor": null,
  "limit": 25,
  "sortKey": "created_at"
} }
{ "id": 20, "result": {
  "data": [
    { "id": "thr_a", "preview": "Create a TUI", "ephemeral": false, "section": { "id": "01984de2-8f74-
7c91-a3b2-5c5e937cf318", "name": "Pinned", "appearance": null }, "modelProvider": "openai",
"createdAt": 1730831111, "updatedAt": 1730831111, "name": "TUI prototype", "status": { "type":
"notLoaded" } },
    { "id": "thr_b", "preview": "Fix tests", "ephemeral": false, "section": null, "modelProvider":
"openai", "createdAt": 1730750000, "updatedAt": 1730750000, "status": { "type": "notLoaded" } }
  ],
  "nextCursor": "opaque-token-or-null"
} }
```

当 `nextCursor` 为 `null` 时，表示你已到达最后一页。

将线程组织到 section 中

使用 `threadSection/create` 创建具名 section，并在 `thread/section/move` 中使用返回的 `section.id`。传入 `beforeThreadId` 可将该线程插入 section 中另一线程之前，省略它则追加到末尾。传入 `sectionId: null` 可将线程移出其 section。使用 `threadSection/list`、`threadSection/update` 和 `threadSection/delete` 来翻页浏览、重命名和删除自定义 section。要固定（pin）某个线程，可将其移动到 `threadSection/list` 返回的内置 Pinned section。

管理线程附件

使用附件（attachment）将 pull request 等资源与已存储的线程关联起来。你可以在不恢复线程的情况下管理这些引用。

- `thread/attachment/add` 接收 `threadId`、`attachmentType`、`identityKey` 和一个 JSON `payload`。对同一线程再次添加相同的类型和 key 时，会返回已存在的附件。
- `thread/attachment/list` 对线程的附件分页。要查找引用某个资源的线程，请携带该资源的 `attachmentType` 和 `identityKey` 调用 `thread/attachmentOwner/list`。
- `thread/attachment/remove` 接收 `threadId`、`attachmentType` 和 `identityKey`。底层资源仍保持可用。
- `thread/attachment/updated` 在关联发生变化时报告 `created` 或 `deleted` 操作。

更新已存储的线程元数据

使用 `thread/metadata/update` 可在不恢复线程的情况下修补已存储的线程元数据。在 `gitInfo` 中传入你想要更新的 Git 字段。省略某个字段以保留其值，或传入 `null` 以清除它。

```
{ "method": "thread/metadata/update", "id": 21, "params": {
  "threadId": "thr_123",
  "gitInfo": { "branch": "feature/sidebar-pr" }
} }
{ "id": 21, "result": {
  "thread": {
    "id": "thr_123",
    "gitInfo": { "sha": null, "branch": "feature/sidebar-pr", "originUrl": null }
  }
} }
```

跟踪线程状态变化

每当已加载线程的运行时状态发生变化时，就会发出 `thread/status/changed`。payload 包含 `threadId` 和新的 `status`。

```
{
  "method": "thread/status/changed",
  "params": {
    "threadId": "thr_123",
    "status": { "type": "active", "activeFlags": ["waitingOnApproval"] }
  }
}
```

列出已加载的线程

`thread/loaded/list` 返回当前加载在内存中的线程 ID。

```
{ "method": "thread/loaded/list", "id": 21 }
{ "id": 21, "result": { "data": ["thr_123", "thr_456"] } }
```

退订已加载的线程

`thread/unsubscribe` 移除当前连接对某个线程的订阅。响应状态为以下之一：

- `unsubscribed`：连接此前已订阅，现已被移除。
- `notSubscribed`：连接此前未订阅该线程。
- `notLoaded`：线程未加载。

在最后一个订阅者离开后，app-server 默认在线程闲置 60 秒后将其卸载。可设置 `thread_unload_delay_secs` 来更改该间隔。卸载线程时，app-server 会发出状态为 `notLoaded` 的 `thread/status/changed` 以及一条 `thread/closed` 通知。

```
{ "method": "thread/unsubscribe", "id": 22, "params": { "threadId": "thr_123" } }
{ "id": 22, "result": { "status": "unsubscribed" } }
```

如果该线程随后过期：

```
{ "method": "thread/status/changed", "params": {
    "threadId": "thr_123",
    "status": { "type": "notLoaded" }
```

```
} }
{ "method": "thread/closed", "params": { "threadId": "thr_123" } }
```

归档线程

使用 `thread/archive` 将已持久化的线程日志（以 JSONL 文件形式存储在磁盘上）移入已归档会话目录。归档线程时还会尝试归档那些尚未归档的派生后代线程。

```
{ "method": "thread/archive", "id": 22, "params": { "threadId": "thr_b" } }
{ "id": 22, "result": {} }
{ "method": "thread/archived", "params": { "threadId": "thr_b" } }
{ "method": "thread/archived", "params": { "threadId": "thr_child" } }
```

已归档的线程不会出现在后续的 `thread/list` 调用中，除非你传入 `archived: true`。服务器会为它实际归档的每个线程发出一条 `thread/archived` 通知；如果某个派生后代无法归档，请求仍可能成功，只是不会发出该后代的归档通知。

删除线程

使用 `thread/delete` 永久删除已持久化的活跃或已归档线程及其派生后代线程。服务器会在返回成功之前移除现有的 rollout 文件和关联元数据；缺失的 rollout 文件会被视为已删除。Ephemeral 根线程无法删除。

```
{ "method": "thread/delete", "id": 23, "params": { "threadId": "thr_b" } }
{ "id": 23, "result": {} }
{ "method": "thread/deleted", "params": { "threadId": "thr_b" } }
{ "method": "thread/deleted", "params": { "threadId": "thr_child" } }
```

取消归档线程

使用 `thread/unarchive` 将已归档线程的 rollout 移回活跃会话目录。

```
{ "method": "thread/unarchive", "id": 24, "params": { "threadId": "thr_b" } }
{ "id": 24, "result": { "thread": { "id": "thr_b", "name": "Bug bash notes" } } }
{ "method": "thread/unarchived", "params": { "threadId": "thr_b" } }
```

触发线程压缩

使用 `thread/compact/start` 为线程触发手动的历史压缩（compaction）。该请求立即返回 `{}`。

App-server 通过同一 `threadId` 上的标准 `turn/*` 和 `item/*` 通知来发出进度，其中包括一个 `contextCompaction` 条目生命周期（先是 `item/started`，然后是 `item/completed`）。

```
{ "method": "thread/compact/start", "id": 25, "params": { "threadId": "thr_b" } }
{ "id": 25, "result": {} }
```

运行线程 shell 命令

使用 `thread/shellCommand` 执行属于某个线程的、由用户发起的 shell 命令。该请求立即返回 `{}`，进度则通过标准的 `turn/*` 和 `item/*` 通知流式传输。

此 API 在沙箱之外运行、拥有完全访问权限，且不继承线程的沙箱策略。客户端应仅对用户显式发起的命令开放此 API。

如果线程已有活跃轮次，该命令会作为该轮次上的辅助操作运行，其格式化输出会被注入到该轮次的消息流中。如果线程处于空闲状态，app-server 会为该 shell 命令启动一个独立轮次。

设置 `timeoutMs` 以毫秒为单位限制执行时间。省略它或传入 `null` 时使用一小时的默认值。`0` 表示请求立即超时；负值会被拒绝。该超时不会延迟立即返回的 RPC 确认。

```
{ "method": "thread/shellCommand", "id": 26, "params": { "threadId": "thr_b", "command": "git status --
short", "timeoutMs": 10000 } }
{ "id": 26, "result": {} }
```

清理后台终端

使用 `thread/backgroundTerminals/clean` 停止与某个线程关联的所有正在运行的后台终端。此方法为实验性，要求 `capabilities.experimentalApi = true`。

```
{ "method": "thread/backgroundTerminals/clean", "id": 27, "params": { "threadId": "thr_b" } }
{ "id": 27, "result": {} }
```

使用 `thread/backgroundTerminals/list` 查看某个已加载线程正在运行的后台终端。该请求支持标准的 `cursor` 和 `limit` 分页，返回的 `processId` 是 app-server 进程 id。此方法为实验性，要求 `capabilities.experimentalApi = true`：

```
{ "method": "thread/backgroundTerminals/list", "id": 28, "params": { "threadId": "thr_b" } }
{ "id": 28, "result": { "data": [
  {
    "itemId": "item_456",
    "processId": "42",
    "command": "python3 -m http.server",
    "cwd": "/workspace",
    "osPid": null,
    "cpuPercent": null,
    "rssKb": null
  }
], "nextCursor": null } }
```

使用 `thread/backgroundTerminals/terminate` 并携带该 `processId` 来停止单个后台终端。此方法为实验性，要求 `capabilities.experimentalApi = true`：

```
{ "method": "thread/backgroundTerminals/terminate", "id": 29, "params": { "threadId": "thr_b",
"processId": "42" } }
{ "id": 29, "result": { "terminated": true } }
```

回退线程历史

使用 `thread/revert` 从持久化的分页线程对话历史中移除 `beforeTurnId` 及其之后的所有轮次。它不会回退本地文件改动。如果线程尚未加载，请先恢复该线程。不支持 ephemeral 线程和具有旧式历史的线程。

```
{ "method": "thread/revert", "id": 30, "params": { "threadId": "thr_b", "beforeTurnId": "turn_456" } }
{ "id": 30, "result": {
  "thread": { "id": "thr_b", "ephemeral": false, "turns": [] },
  "turnsBackwardsCursor": "retained-turn-cursor-or-null",
  "itemsBackwardsCursor": "retained-item-cursor-or-null"
} }
{ "method": "thread/reverted", "params": { "threadId": "thr_b" } }
```

返回的 `thread.turns` 始终为空。将返回的 cursor 与 `thread/turns/list` 或 `thread/items/list` 以及 `sortDirection: "desc"` 一起使用，即可加载保留的历史，包括该 cursor 的锚点。

`thread/rollback` 已被移除，调用会返回 unknown-method 错误。旧保存线程中已有的 rollback 标记仍然可以读取。

轮次（Turns）

`input` 字段接受一个项（item）列表：

- `{ "type": "text", "text": "Explain this diff" }`
- `{ "type": "image", "url": "https://.../design.png" }`
- `{ "type": "localImage", "path": "/tmp/screenshot.png" }`

你可以按轮次（turn）覆盖配置设置（模型、effort、`cwd`、沙箱策略、摘要）。指定后，这些设置会成为同一线程（thread）中后续轮次的默认值。`outputSchema` 仅作用于当前轮次。对于 `sandboxPolicy.type = "externalSandbox"`，需将 `networkAccess` 设为 `restricted` 或 `enabled`；对于 `workspaceWrite`，`networkAccess` 仍为布尔值。

对于 `turn/start.collaborationMode`，`settings.developer_instructions: null` 表示“使用所选模式的内置指令”，而不是清除模式指令。

沙箱读取访问（`ReadOnlyAccess`）

`sandboxPolicy` 支持显式的读取访问控制：

- `readOnly`：可选的 `access`（默认为 `{ "type": "fullAccess" }`，或为受限根目录）。
- `workspaceWrite`：可选的 `readOnlyAccess`（默认为 `{ "type": "fullAccess" }`，或为受限根目录）。

受限读取访问的结构：

```
{
  "type": "restricted",
  "includePlatformDefaults": true,
  "readableRoots": ["/Users/me/shared-read-only"]
}
```

在 macOS 上，`includePlatformDefaults: true` 会为受限读取会话附加一份经过筛选的平台默认 Seatbelt 策略。这样可以在不宽泛放开整个 `/System` 的情况下提升工具兼容性。

示例：

```
{ "type": "readOnly", "access": { "type": "fullAccess" } }
```

```
{
  "type": "workspaceWrite",
  "writableRoots": ["/Users/me/project"],
  "readOnlyAccess": {
    "type": "restricted",
    "includePlatformDefaults": true,
    "readableRoots": ["/Users/me/shared-read-only"]
  },
  "networkAccess": false
}
```

启动轮次

```
{ "method": "turn/start", "id": 30, "params": {
  "threadId": "thr_123",
  "input": [ { "type": "text", "text": "Run tests" } ],
  "cwd": "/Users/me/project",
  "approvalPolicy": "on-request",
  "sandboxPolicy": {
    "type": "workspaceWrite",
    "writableRoots": ["/Users/me/project"],
    "networkAccess": true
  },
  "model": "gpt-6.1-sol",
  "effort": "medium",
  "summary": "concise",
  "outputSchema": {
    "type": "object",
    "properties": { "answer": { "type": "string" } },
    "required": ["answer"],
    "additionalProperties": false
  }
} }
{ "id": 30, "result": { "turn": { "id": "turn_456", "status": "inProgress", "items": [], "error": null
} } }
```

设置 `serviceTierForTurn` 可为某个新轮次覆盖服务层级。使用 `"default"` 表示标准速度，或省略该字段以继承线程的服务层级。若要为当前轮次及其后的轮次更改服务层级，请设置 `serviceTier`。

要以你的客户端所运行工具的输出启动一个轮次，请传入 `toolOutput`，其中包含非空的 `name`、可选的 `namespace`，以及一个 `output` 字符串或内容项数组。将 `input` 设为空数组；不能将 `toolOutput` 与非空的用户输入组合使用。

```
{
  "method": "turn/start",
  "id": 31,
  "params": {
    "threadId": "thr_123",
    "input": [],
    "toolOutput": {
```

```
      "name": "run_tests",
      "namespace": null,
      "output": "All 42 tests passed."
    }
  }
}
```

该输出在对话中仍作为工具输出，并以 `functionCallOutput` 项的形式出现在通知和持久化历史中。如果某个常规轮次已处于活动状态，Codex 会将该输出排入该轮次的队列。

向线程注入项

使用 `thread/inject_items` 可将预构建的 Responses API 项追加到已加载线程的 prompt 历史中，而无需启动用户轮次。这些项会被持久化到 rollout，并包含在后续的模型请求中。

```
{ "method": "thread/inject_items", "id": 31, "params": {
  "threadId": "thr_123",
  "items": [
    {
      "type": "message",
      "role": "assistant",
      "content": [{ "type": "output_text", "text": "Previously computed context." }]
    }
  ]
} }
{ "id": 31, "result": {} }
```

转向活动轮次

使用 `turn/steer` 向正在执行的活动轮次追加更多用户输入。

- 包含 `expectedTurnId`；它必须与活动轮次的 ID 匹配。
- 如果线程上没有活动的轮次，请求会失败。
- `turn/steer` 不会发出新的 `turn/started` 通知。
- `turn/steer` 不接受轮次级别的覆盖项（`model`、`cwd`、`sandboxPolicy` 或 `outputSchema`）。

```
{ "method": "turn/steer", "id": 32, "params": {
  "threadId": "thr_123",
  "input": [ { "type": "text", "text": "Actually focus on failing tests first." } ],
  "expectedTurnId": "turn_456"
} }
{ "id": 32, "result": { "turnId": "turn_456" } }
```

启动轮次（调用技能）

通过在文本输入中包含 `$<skill-name>` 并同时添加一个 `skill` 输入项，可显式调用技能。

```
{ "method": "turn/start", "id": 33, "params": {
  "threadId": "thr_123",
  "input": [
    { "type": "text", "text": "$skill-creator Add a new skill for triaging flaky CI and include step-
by-step usage." },
    { "type": "skill", "name": "skill-creator", "path": "/Users/me/.codex/skills/skill-
```

```
creator/SKILL.md" }
  ]
} }
{ "id": 33, "result": { "turn": { "id": "turn_457", "status": "inProgress", "items": [], "error": null
} } }
```

中断轮次

```
{ "method": "turn/interrupt", "id": 31, "params": { "threadId": "thr_123", "turnId": "turn_456" } }
{ "id": 31, "result": {} }
```

成功时，该轮次以 `status: "interrupted"` 结束。

审查

`review/start` 会为线程运行 Codex 审查器，并流式传输审查项。目标包括：

- `uncommittedChanges`
- `baseBranch`（与某个分支做 diff）
- `commit`（审查特定提交）
- `custom`（自由格式的指令）

`review/start` 会在所提供的线程上运行审查。可以省略 `delivery`，或传入 `"inline"`。若要运行一次独立的审查，可使用 `thread/fork` 以包含既有历史，或使用 `thread/start` 创建全新线程，然后用新的线程 ID 调用 `review/start`。

请求/响应示例：

```
{ "method": "review/start", "id": 40, "params": {
  "threadId": "thr_123",
  "delivery": "inline",
  "target": { "type": "commit", "sha": "1234567deadbeef", "title": "Polish tui colors" }
} }
{ "id": 40, "result": {
  "turn": {
    "id": "turn_900",
    "status": "inProgress",
    "items": [
      { "type": "userMessage", "id": "turn_900", "content": [ { "type": "text", "text": "Review commit
1234567: Polish tui colors" } ] }
    ],
    "error": null
  },
  "reviewThreadId": "thr_123"
} }
```

Codex 会照常流式发送 `turn/started` 通知，随后是一条带有 `enteredReviewMode` 项的 `item/started`：

```
{
  "method": "item/started",
```

```
  "params": {
    "item": {
      "type": "enteredReviewMode",
      "id": "turn_900",
      "review": "current changes"
    }
  }
}
```

当审查器完成时，服务器会发出 `item/started` 和 `item/completed`，其中包含带有最终审查文本的 `exitedReviewMode` 项：

```
{
  "method": "item/completed",
  "params": {
    "item": {
      "type": "exitedReviewMode",
      "id": "turn_900",
      "review": "Looks solid overall..."
    }
  }
}
```

使用此通知在你的客户端中渲染审查器输出。

进程执行

`process/*` 是一个实验性的显式进程控制 API。它需要 `capabilities.experimentalApi = true`，并且运行在 Codex 的沙箱之外。仅当你的客户端有意在无沙箱的情况下暴露本地进程控制时才使用它。

使用 `process/spawn` 启动进程并提供一个 `processHandle`，然后将该句柄用于 stdin、调整大小和终止请求。输出通过 `process/outputDelta` 通知流式传输，完成情况则通过 `process/exited` 流式传输。

```
{ "method": "process/spawn", "id": 48, "params": {
  "command": ["python3", "-m", "pytest", "-q"],
  "processHandle": "pytest-1",
  "cwd": "/Users/me/project",
  "tty": true
} }
{ "id": 48, "result": {} }
{ "method": "process/outputDelta", "params": {
  "processHandle": "pytest-1",
  "stream": "stdout",
  "deltaBase64": "Li4u"
} }
{ "method": "process/exited", "params": {
  "processHandle": "pytest-1",
  "exitCode": 0
} }
```

使用 `process/writeStdin` 并配合 `deltaBase64`、`closeStdin` 或两者兼用来发送输入。使用 `process/resizePty` 处理 PTY 大小调整事件，使用 `process/kill` 终止正在运行的进程。

命令执行

`command/exec` 在服务器沙箱下运行单条命令（`argv` 数组），而不创建线程。

```
{ "method": "command/exec", "id": 50, "params": {
  "command": ["ls", "-la"],
  "cwd": "/Users/me/project",
  "sandboxPolicy": { "type": "workspaceWrite" },
  "timeoutMs": 10000
} }
{ "id": 50, "result": { "exitCode": 0, "stdout": "...", "stderr": "" } }
```

如果你已经对服务器进程做了沙箱隔离，并希望 Codex 跳过其自身的沙箱强制执行，可使用 `sandboxPolicy.type = "externalSandbox"`。对于外部沙箱模式，将 `networkAccess` 设为 `restricted`（默认）或 `enabled`。对于 `readOnly` 和 `workspaceWrite`，可使用与上面所示相同的可选 `access` / `readOnlyAccess` 结构。

注意：

- 服务器会拒绝空的 `command` 数组。
- `sandboxPolicy` 接受与 `turn/start` 所用相同的结构（例如 `dangerFullAccess`、`readOnly`、`workspaceWrite`、`externalSandbox`）。
- 省略时，`timeoutMs` 会回退到服务器默认值。
- 为基于 PTY 的会话设置 `tty: true`；当你计划随后使用 `command/exec/write`、`command/exec/resize` 或 `command/exec/terminate` 时，请使用 `processId`。
- 设置 `streamStdoutStderr: true`，以便在命令运行期间接收 `command/exec/outputDelta` 通知。

读取管理要求（`configRequirements/read`）

使用 `configRequirements/read` 可查看从 `requirements.toml` 和/或 MDM 加载的生效管理要求。

```
{ "method": "configRequirements/read", "id": 52, "params": {} }
{ "id": 52, "result": {
  "requirements": {
    "allowedApprovalPolicies": ["on-request", "never"],
    "allowedSandboxModes": ["read-only", "workspace-write"],
    "featureRequirements": {
      "unified_exec": false
    }
  }
} }
```

未配置任何要求时，`result.requirements` 为 `null`。当 `result.supportsIndependentSpeedModes` 为 `true` 时，`fast_mode` 和 `ultrafast_mode` 要求各自独立生效。较旧的服务器会省略该字段，并对两者都使用 `fast_mode`。有关受支持的键和值，请参阅 `requirements.toml` 参考。

Windows 沙箱设置（`windowsSandbox/setupStart`）

在提示进行设置之前，请先调用 `windowsSandbox/readiness`。MXC 和已配置的旧版沙箱都可能返回 `ready`。如果旧版沙箱返回 `updateRequired`，请调用 `windowsSandbox/setupStart` 以异步方式启动设置。如果 Windows 沙箱被禁用，或主机运行的是其他操作系统，就绪检查会返回 `notConfigured`。

```
{ "method": "windowsSandbox/setupStart", "id": 53, "params": { "mode": "elevated" } }
{ "id": 53, "result": { "started": true } }
```

App-server 会在后台启动设置，随后发出一条完成通知：

```
{
  "method": "windowsSandbox/setupCompleted",
  "params": { "mode": "elevated", "success": true, "error": null }
}
```

模式：

- `elevated` —— 运行提权的 Windows 沙箱设置路径。
- `unelevated` —— 运行旧版的设置/预检路径。

文件系统

v2 文件系统 API 基于绝对路径操作。当客户端需要在文件或目录发生变化后使 UI 状态失效时，可使用 `fs/watch`。

```
{ "method": "fs/watch", "id": 54, "params": {
  "watchId": "0195ec6b-1d6f-7c2e-8c7a-56f2c4a8b9d1",
  "path": "/Users/me/project/.git/HEAD"
} }
{ "id": 54, "result": { "path": "/Users/me/project/.git/HEAD" } }
{ "method": "fs/changed", "params": {
  "watchId": "0195ec6b-1d6f-7c2e-8c7a-56f2c4a8b9d1",
  "changedPaths": ["/Users/me/project/.git/HEAD"]
} }
{ "method": "fs/unwatch", "id": 55, "params": {
  "watchId": "0195ec6b-1d6f-7c2e-8c7a-56f2c4a8b9d1"
} }
{ "id": 55, "result": {} }
```

监视某个文件会针对该文件路径发出 `fs/changed`，包括通过替换或重命名操作带来的更新。

事件

事件通知是服务器发起的流，涵盖线程生命周期、轮次生命周期以及其中的各个项。启动或恢复线程之后，应持续读取活动的传输流，以获取 `thread/started`、`thread/archived`、`thread/unarchived`、`thread/closed`、`thread/status/changed`、`turn/*`、`item/*` 和 `serverRequest/resolved` 通知。

通知选择退出（opt-out）

客户端可以通过在 `initialize.params.capabilities.optOutNotificationMethods` 中发送精确的方法名，按连接屏蔽特定通知。

- 仅精确匹配：`item/agentMessage/delta` 只屏蔽该方法。
- 未知的方法名会被忽略。
- 适用于当前的 `thread/*`、`turn/*`、`item/*` 以及相关的 v2 通知。
- 不适用于请求、响应或错误。

模糊文件搜索事件（实验性）

模糊文件搜索会话 API 会按查询发出通知：

- `fuzzyFileSearch/sessionUpdated` —— `{ sessionId, query, files }`，包含当前查询的最新匹配结果。
- `fuzzyFileSearch/sessionCompleted` —— 该查询的索引和匹配完成后发出 `{ sessionId }`。

警告事件

- `configWarning` —— `{ summary, details?, path?, range? }`，用于可恢复的配置或初始化问题。
- `warning` —— `{ threadId?, message }`，用于非致命的运行时警告。

Windows 沙箱设置事件

- `windowsSandbox/setupCompleted` —— 在 `windowsSandbox/setupStart` 请求结束后发出 `{ mode, success, error }`。

轮次事件

- `turn/started` —— `{ turn }`，包含轮次 ID、空的 `items` 以及 `status: "inProgress"`。
- `turn/completed` —— `{ turn }`，其中 `turn.status` 为 `completed`、`interrupted` 或 `failed`；失败时会携带 `{ error: { message, codexErrorInfo?, additionalDetails? } }`。
- `turn/diff/updated` —— `{ threadId, turnId, diff }`，包含轮次内所有文件变更的最新聚合统一 diff。
- `turn/plan/updated` —— 每当智能体（agent）分享或更改其计划时发出 `{ turnId, explanation?, plan }`；每个 `plan` 条目为 `{ step, status }`，其中 `status` 为 `pending`、`inProgress` 或 `completed`。
- `hook/started` 和 `hook/completed` —— 同步生命周期钩子启动时，以及其最终运行摘要可用时，发出 `{ threadId, turnId?, run }`。异步钩子不会发出这些通知。
- `model/safetyBuffering/updated` —— 当响应进入瞬态安全缓冲时发出 `{ threadId, turnId, model, useCases, reasons, showBufferingUi, fasterModel }`。
- `model/rerouted` —— 当服务将请求路由到另一个模型时发出 `{ threadId, turnId, fromModel, toModel, reason }`。

- `model/verification` —— 当服务要求额外的账户验证时发出 `{ threadId, turnId, verifications }`。
- `thread/tokenUsage/updated` —— 活动线程的 token 用量更新。

`turn/diff/updated` 和 `turn/plan/updated` 目前即使在项事件流式传输时也会包含空的 `items` 数组。请将 `item/*` 通知作为轮次项的权威来源。

项

`ThreadItem` 是轮次响应和 `item/*` 通知中携带的标签联合类型（tagged union）。常见的项类型包括：

- `userMessage` —— `{id, content}`，其中 `content` 是一个用户输入列表（`text`、`image` 或 `localImage`）。
- `functionCallOutput` —— `{id, name, namespace, output}`，用于通过 `turn/start.toolOutput` 提供的独立工具输出。`namespace` 可以为 `null`。
- `agentMessage` —— `{id, text, phase?}`，包含累积的智能体回复。存在时，`phase` 使用 Responses API 的线上传输值（`commentary`、`partial_answer`、`final_answer`）。`partial_answer` 是稳定的答案文本，其后可能还有更多输出或工具调用。
- `plan` —— `{id, text}`，包含计划模式下提出的计划文本。请以 `item/completed` 中的最终 `plan` 项为准。
- `reasoning` —— `{id, summary, content}`，其中 `summary` 保存流式传输的推理摘要，`content` 保存原始推理块。
- `commandExecution` —— `{id, command, cwd, status, commandActions, aggregatedOutput?, exitCode?, durationMs?}`。
- `fileChange` —— `{id, changes, status}`，描述提议的编辑；`changes` 列出 `{path, kind, diff}`。
- `mcpToolCall` —— 记录调用的 `id`、`server`、`tool`、`status` 和 `arguments`。 该调用还包含 `appContext`、`mcpAppUi`、`pluginId`、`readOnlyHint`、`result`、`error` 和 `durationMs`。这些字段均可以为 `null`。旧版 `mcpAppResourceUri` 字段为可选。 对于受信任的 MCP 应用，`appContext` 包含 `connectorId`。它的其他字段（`linkId`、`resourceUri`、`appName` 和 `actionName`）可以为 `null`。 当工具描述符指定了资源 URI 和显示模式（`inline` 或 `fullscreen`）时，`mcpAppUi` 包含 `resourceUri` 和 `preferredModelDisplayMode`。对于未保存此展示元数据的调用，app-server 会为 `mcpAppUi` 返回 `null`。
- `dynamicToolCall` —— `{id, tool, arguments, status, contentItems?, success?, durationMs?}`，用于由客户端执行的动态工具调用。
- `collabToolCall` —— `{id, tool, status, senderThreadId, receiverThreadId?, newThreadId?, prompt?, agentStatus?}`。
- `webSearch` —— `{id, query, action?}`，用于智能体发出的网页搜索请求。
- `imageView` —— 智能体调用图片查看器工具时发出 `{id, path}`。
- `enteredReviewMode` —— 审查器启动时发送 `{id, review}`。
- `exitedReviewMode` —— 审查器完成时发出 `{id, review}`。
- `contextCompaction` —— 当 Codex 压缩（compaction）对话历史时发出 `{id}`。

对于 `webSearch.action`，动作 `type` 可以为 `search`（`query?`、`queries?`）、`openPage`（`url?`）或 `findInPage`（`url?`、`pattern?`）。

app server 已弃用旧版的 `thread/compacted` 通知；请改用 `contextCompaction` 项。

所有项都会发出两个共享的生命周期事件：

- `item/started` —— 在新的工作单元开始时发出完整的 `item`；`item.id` 与增量所使用的 `itemId` 一致。
- `item/completed` —— 在工作结束时发送最终的 `item`；请将其视为权威状态。

项增量

- `item/agentMessage/delta` —— 为智能体消息追加流式文本。
- `item/plan/delta` —— 流式传输提议的计划文本。最终的 `plan` 项可能与拼接后的增量不完全一致。
- `item/reasoning/summaryTextDelta` —— 流式传输可读的推理摘要；当新的摘要小节开启时，`summaryIndex` 会递增。
- `item/reasoning/summaryPartAdded` —— 标记推理摘要小节之间的边界。
- `item/reasoning/textDelta` —— 流式传输原始推理文本（在模型支持时）。
- `item/commandExecution/outputDelta` —— 流式传输命令的 stdout/stderr；请按顺序追加各增量。
- `item/fileChange/outputDelta` —— 已弃用的兼容性通知，用于旧版 `apply_patch` 文本输出。当前版本的 app-server 不再发出该通知；请改用 `fileChange` 项和 `turn/diff/updated`。

错误

当 app-server 在优雅关闭期间拒绝新工作时，会返回一个 JSON-RPC 错误，其错误码为 `-32600`，且 `data.reason` 为 `serverShuttingDown`。

`error` 通知包含一个嵌套的 `error` 对象，其中有 `message` 和可选的 `additionalDetails`。其 `codexErrorInfo` 字段可以为 `null`。如果 `willRetry` 为 `true`，服务器会重试，且该轮次保持活动状态。失败的轮次以 `status: "failed"` 结束。常见的 `codexErrorInfo` 值包括 `contextWindowExceeded`、`usageLimitExceeded`、`badRequest`、`unauthorized` 和 `sandboxError`。对于上游请求失败，该值可以是诸如 `{ "httpConnectionFailed": { "httpStatusCode": 503 } }` 的对象。

审批

取决于用户的 Codex 设置，命令执行和文件更改可能需要审批。app-server 会向客户端发送一个由服务器发起的 JSON-RPC 请求，客户端则以一个决策负载进行响应。

- 命令执行决策：`accept`、`acceptForSession`、`decline`、`cancel` 或 `{ "acceptWithExecpolicyAmendment": { "execpolicy_amendment": ["cmd", "..."] } }`。
- 文件更改决策：`accept`、`acceptForSession`、`decline`、`cancel`。

- 请求包含 `threadId` 和 `turnId` —— 用它们将 UI 状态限定到当前活动对话。
- 服务器会恢复或拒绝该工作，并以 `item/completed` 结束该项。

命令执行审批

消息顺序：

1. `item/started` 显示处于等待状态的 `commandExecution` 项，其中包含 `command`、`cwd` 和其他字段。
2. `item/commandExecution/requestApproval` 包含 `itemId`、`threadId`、`turnId`、可选的 `reason`、可选的 `command`、可选的 `cwd`、可选的 `commandActions`、可选的 `proposedExecpolicyAmendment`、可选的 `networkApprovalContext` 以及可选的 `availableDecisions`。当 `initialize.params.capabilities.experimentalApi = true` 时，负载还可以包含实验性的 `additionalPermissions`，用于描述所请求的逐命令沙箱访问权限。`additionalPermissions` 中的任何文件系统路径在传输时均为绝对路径。
3. 客户端使用上述命令执行审批决策之一进行响应。
4. `serverRequest/resolved` 确认等待中的请求已被应答或清除。
5. `item/completed` 返回最终的 `commandExecution` 项，其 `status` 为 `completed | failed | declined`。

当存在 `networkApprovalContext` 时，该提示针对的是受管理的网络访问（而不是一般的 shell 命令审批）。当前的 v2 schema 会公开目标 `host` 和 `protocol`；客户端应渲染针对网络的提示，而不应依赖 `command` 作为对用户有意义的 shell 命令预览。

Codex 会按目标（`host`、协议和端口）对并发的网络审批提示进行分组。因此，app-server 可能会发送一个提示来同时放行多个发往同一目标的排队请求，而同一主机上的不同端口会被单独处理。

文件更改审批

消息顺序：

1. `item/started` 发出一个 `fileChange` 项，其中包含提议的 `changes` 和 `status: "inProgress"`。
2. `item/fileChange/requestApproval` 包含 `itemId`、`threadId`、`turnId`、可选的 `reason` 以及可选的 `grantRoot`。
3. 客户端使用上述文件更改审批决策之一进行响应。
4. `serverRequest/resolved` 确认等待中的请求已被应答或清除。
5. `item/completed` 返回最终的 `fileChange` 项，其 `status` 为 `completed | failed | declined`。

`tool/requestUserInput`

当客户端响应 `item/tool/requestUserInput` 时，app-server 会发出带有 `{ threadId, requestId }` 的 `serverRequest/resolved`。如果在客户端应答之前，等待中的请求因轮次启动、轮次完成或轮次中断而被清除，服务器会针对该清理发出同样的通知。

请求参数包含 `autoResolutionMs`，它是一个整数毫秒超时值或 `null`。存在该参数时，如果用户没有应答，宿主客户端可以在该时间间隔后自动应答该提示。

权限请求

内置的 `request_permissions` 工具会发送 `item/permissions/requestApproval`，其中包含 `threadId`、`turnId`、`itemId`、`environmentId`、`cwd`、可选的 `reason`，以及所请求的网络或文件系统权限。响应时请使用 `permissions`，且其中仅包含被授予的子集。将 `scope` 设为 `"session"` 可将授权持久化到同一会话中的后续轮次；省略它或使用 `"turn"` 则为轮次范围的授权。未被请求的权限会被忽略。

MCP 服务器引导（elicitation）请求

MCP 服务器可以通过 `mcpServer/elicitation/request` 中断一个轮次。该请求包含 `threadId`、可选的 `turnId`、`serverName`，以及以下请求形式之一：

- `mode: "form"` 或 `mode: "openai/form"`，附带 `message` 和 `requestedSchema`。
- `mode: "url"`，附带 `message`、`url` 和 `elicitationId`。

以 `action: "accept"` 和所请求的 `content` 进行响应，或以 `action: "decline"` 或 `"cancel"` 以及 `content: null` 进行响应。随后 app-server 会发出 `serverRequest/resolved`。若要接收 `openai/form` 变体，请通过 `initialize.params.capabilities.mcpServerOpenaiFormElicitation` 选择启用。

动态工具调用（实验性）

`thread/start` 上的 `dynamicTools` 以及相应的 `item/tool/call` 请求或响应流均为实验性 API。

动态工具名和命名空间名必须遵循 Responses API 的命名约束。请避免使用 Codex 内置工具所用的保留命名空间名。

当在轮次期间调用动态工具时，app-server 会发出：

1. `item/started`，其中 `item.type = "dynamicToolCall"`、`status = "inProgress"`，外加 `tool` 和 `arguments`。
2. `item/tool/call`，作为发送给客户端的服务器请求。
3. 客户端响应负载，其中包含返回的内容项。
4. `item/completed`，其中 `item.type = "dynamicToolCall"`、最终的 `status`，以及任何返回的 `contentItems` 或 `success` 值。

MCP 工具调用审批（应用）

应用（连接器）工具调用也可能需要审批。当应用工具调用具有副作用时，服务器可以通过 `tool/requestUserInput` 征求审批，选项包括**接受（Accept）**、**拒绝（Decline）**和**取消（Cancel）**。破坏性工具注解始终会触发审批，即使该工具还宣告了权限更低的提示。如果用户拒绝或取消，相关的 `mcpToolCall` 项会以错误结束，而不会运行该工具。

技能（Skills）

在用户文本输入中包含 `$<skill-name>` 即可调用某个技能。再添加一个 `skill` 输入项（推荐），让服务器注入完整的技能指令，而不是依赖模型去解析该名称。

```
{
  "method": "turn/start",
  "id": 101,
  "params": {
    "threadId": "thread-1",
    "input": [
      {
        "type": "text",
        "text": "$skill-creator Add a new skill for triaging flaky CI."
      },
      {
        "type": "skill",
        "name": "skill-creator",
        "path": "/Users/me/.codex/skills/skill-creator/SKILL.md"
      }
    ]
  }
}
```

如果省略 `skill` 项，模型仍会解析 `$<skill-name>` 标记并尝试定位该技能，但这可能增加延迟。

示例：

```
$skill-creator Add a new skill for triaging flaky CI and include step-by-step usage.
```

使用 `skills/list` 获取可用技能（可选择用 `cwds` 限定范围，并配合 `forceReload`）。你还可以包含 `perCwdExtraUserRoots`，为特定的 `cwd` 值把额外的绝对路径作为 `user` 作用域来扫描。App-server 会忽略其 `cwd` 未出现在 `cwds` 中的条目。`skills/list` 可能会复用按 `cwd` 缓存的结果；设置 `forceReload: true` 可从磁盘刷新。存在 `SKILL.json` 时，服务器会从中读取 `interface` 和 `dependencies`。

```
{ "method": "skills/list", "id": 25, "params": {
  "cwds": ["/Users/me/project", "/Users/me/other-project"],
  "forceReload": true,
  "perCwdExtraUserRoots": [
    {
      "cwd": "/Users/me/project",
      "extraUserRoots": ["/Users/me/shared-skills"]
    }
  ]
} }
{ "id": 25, "result": {
  "data": [{
    "cwd": "/Users/me/project",
    "skills": [
      {
        "name": "skill-creator",
        "description": "Create or update a Codex skill",
        "enabled": true,
        "interface": {
```

```
          "displayName": "Skill Creator",
          "shortDescription": "Create or update a Codex skill"
        },
        "dependencies": {
          "tools": [
            {
              "type": "env_var",
              "value": "GITHUB_TOKEN",
              "description": "GitHub API token"
            },
            {
              "type": "mcp",
              "value": "github",
              "transport": "streamable_http",
              "url": "https://example.com/mcp"
            }
          ]
        }
      }
    ],
    "errors": []
  }]
} }
```

当被监视的本地技能文件发生变化时，服务器还会发出 `skills/changed` 通知。请将其视为失效信号，并在需要时用当前参数重新运行 `skills/list`。

按路径启用或禁用某个技能：

```
{
  "method": "skills/config/write",
  "id": 26,
  "params": {
    "path": "/Users/me/.codex/skills/skill-creator/SKILL.md",
    "enabled": false
  }
}
```

应用（Apps）与连接器（connectors）

使用 `app/installed` 读取最新提交的已安装应用运行时快照。每个结果都包含应用 `id`、`runtimeName`（或 `null`）、生效的 `enabled` 状态和 `callable` 状态。只有当生效配置启用了该应用，且至少有一个模型可见的工具符合应用与工具策略时，应用才是可调用的。

```
{
  "method": "app/installed",
  "id": 49,
  "params": {
    "threadId": "thread-1",
    "forceRefresh": false
  }
}
{
  "id": 49,
  "result": {
    "apps": [
```

```
      {
        "id": "demo-app",
        "runtimeName": "Demo App",
        "enabled": true,
        "callable": true
      }
    ]
  }
}
```

省略 `threadId` 可使用全局配置，而不是已加载线程（thread）的配置。设置 `forceRefresh: true` 可在读取前刷新连接器运行时快照。当全局或工作区策略阻止应用访问时，被观测到的应用仍可能出现在结果中，但其 `enabled` 和 `callable` 为 `false`。

使用 `app/list` 获取可用应用。在 CLI/TUI 中，`/apps` 是面向用户的选择器；在自定义客户端中，可直接调用 `app/list`。每个条目都同时包含 `isAccessible`（对用户可用）和 `isEnabled`（在 `config.toml` 中已启用），以便客户端区分安装/访问与本地启用状态。应用条目还可以包含可选的 `branding`、`appMetadata` 和 `labels` 字段。

```
{ "method": "app/list", "id": 50, "params": {
  "cursor": null,
  "limit": 50,
  "threadId": "thread-1",
  "forceRefetch": false
} }
{ "id": 50, "result": {
  "data": [
    {
      "id": "demo-app",
      "name": "Demo App",
      "description": "Example connector for documentation.",
      "logoUrl": "https://example.com/demo-app.png",
      "logoUrlDark": null,
      "distributionChannel": null,
      "branding": null,
      "appMetadata": null,
      "labels": null,
      "installUrl": "https://chatgpt.com/apps/demo-app/demo-app",
      "isAccessible": true,
      "isEnabled": true
    }
  ],
  "nextCursor": null
} }
```

如果提供 `threadId`，应用功能门控（`features.apps`）会使用该线程的配置快照；省略时，app-server 使用最新的全局配置。

`app/list` 会在可访问应用和目录应用都加载完成后才返回。设置 `forceRefetch: true` 可绕过应用缓存获取新数据。只有刷新成功时才会替换缓存条目。

每当任一数据源（可访问应用或目录应用）加载完成时，服务器还会发出 `app/list/updated` 通知。每条通知都包含最新合并后的应用列表。

```
{
  "method": "app/list/updated",
  "params": {
    "data": [
      {
        "id": "demo-app",
        "name": "Demo App",
        "description": "Example connector for documentation.",
        "logoUrl": "https://example.com/demo-app.png",
        "logoUrlDark": null,
        "distributionChannel": null,
        "branding": null,
        "appMetadata": null,
        "labels": null,
        "installUrl": "https://chatgpt.com/apps/demo-app/demo-app",
        "isAccessible": true,
        "isEnabled": true
      }
    ]
  }
}
```

当你已经知道应用 ID、需要的是应用元数据而不是已安装运行时状态时，使用 `app/read`。最多传入 100 个 `appIds`。服务器只保留每个重复 ID 的首次出现，并在 `apps` 和 `missingAppIds` 中保持该顺序。未知或不可访问的应用会在 `missingAppIds` 中返回，而不会让整个请求失败。

```
{
  "method": "app/read",
  "id": 52,
  "params": {
    "appIds": ["demo-app", "missing-app"],
    "includeTools": true
  }
}
{
  "id": 52,
  "result": {
    "apps": [
      {
        "id": "demo-app",
        "name": "Demo App",
        "description": "Example connector for documentation.",
        "iconUrl": null,
        "iconUrlDark": null,
        "distributionChannel": null,
        "installUrl": null,
        "pluginDisplayNames": [],
        "toolSummaries": [
          {
            "name": "search",
            "title": "Search",
            "description": "Search the app.",
            "isEnabled": true,
            "disabledReason": null,
            "isReadOnly": true
          }
        ]
      }
    ],
```

```
    "missingAppIds": ["missing-app"]
  }
}
```

设置 `includeTools: true` 可请求仅供展示的公开工具摘要。元数据响应不包含已安装应用的运行时状态，也不会授权工具调用；请使用 `app/installed` 检查生效的 `enabled` 和 `callable` 状态。

调用应用的方式是：在文本输入中插入 `$<app-slug>`，并添加一个带 `app://<id>` 路径的 `mention` 输入项（推荐）。

```
{
  "method": "turn/start",
  "id": 51,
  "params": {
    "threadId": "thread-1",
    "input": [
      {
        "type": "text",
        "text": "$demo-app Pull the latest updates from the team."
      },
      {
        "type": "mention",
        "name": "Demo App",
        "path": "app://demo-app"
      }
    ]
  }
}
```

应用设置的 Config RPC 示例

使用 `config/read`、`config/value/write` 和 `config/batchWrite` 检查或更新 `config.toml` 中的应用控制项。

读取生效的应用配置结构（包括 `_default` 和针对各工具的覆盖设置）：

```
{ "method": "config/read", "id": 60, "params": { "includeLayers": false } }
{ "id": 60, "result": {
  "config": {
    "apps": {
      "_default": {
        "enabled": true,
        "destructive_enabled": true,
        "open_world_enabled": true,
        "approvals_reviewer": "user",
        "default_tools_approval_mode": "auto"
      },
      "google_drive": {
        "enabled": true,
        "destructive_enabled": false,
        "approvals_reviewer": "auto_review",
        "default_tools_approval_mode": "prompt",
        "tools": {
          "files/delete": { "enabled": false, "approval_mode": "approve" }
        }
      }
```

```
    }
  }
} }
```

`apps._default.approvals_reviewer` 为所有应用设置审批人，除非被某个应用级的值覆盖。两者都省略时，应用会继承顶层的 `approvals_reviewer` 值。
`apps._default.default_tools_approval_mode` 为没有应用级或工具级覆盖的工具设置回退审批模式。托管审批模式要求会覆盖工具审批模式设置。

更新单个应用设置：

```
{
  "method": "config/value/write",
  "id": 61,
  "params": {
    "keyPath": "apps.google_drive.default_tools_approval_mode",
    "value": "prompt",
    "mergeStrategy": "replace"
  }
}
```

原子地应用多个应用配置修改：

```
{
  "method": "config/batchWrite",
  "id": 62,
  "params": {
    "edits": [
      {
        "keyPath": "apps._default.destructive_enabled",
        "value": false,
        "mergeStrategy": "upsert"
      },
      {
        "keyPath": "apps.google_drive.tools.files/delete.approval_mode",
        "value": "approve",
        "mergeStrategy": "upsert"
      }
    ]
  }
}
```

检测并导入外部智能体（agent）配置

使用 `externalAgentConfig/detect` 发现可迁移的外部智能体产物，然后将选中的条目传给 `externalAgentConfig/import`。

检测示例：

```
{ "method": "externalAgentConfig/detect", "id": 63, "params": {
  "includeHome": true,
  "cwds": ["/Users/me/project"]
} }
{ "id": 63, "result": {
```

```
  "items": [
    {
      "itemType": "AGENTS_MD",
      "description": "Import /Users/me/project/CLAUDE.md to /Users/me/project/AGENTS.md.",
      "cwd": "/Users/me/project"
    },
    {
      "itemType": "SKILLS",
      "description": "Copy skill folders from /Users/me/.claude/skills to /Users/me/.agents/skills.",
      "cwd": null
    }
  ]
} }
```

导入示例：

```
{ "method": "externalAgentConfig/import", "id": 64, "params": {
  "migrationItems": [
    {
      "itemType": "AGENTS_MD",
      "description": "Import /Users/me/project/CLAUDE.md to /Users/me/project/AGENTS.md.",
      "cwd": "/Users/me/project"
    }
  ],
  "source": "claude-code"
} }
{ "id": 64, "result": { "importId": "8ae96ff3-3425-4f4c-8772-b6fd61502868" } }
```

可选的顶层 `source` 导入参数用于标注产生所选迁移条目的产品。

服务器会在各条目类型完成时发出 `externalAgentConfig/import/progress`，并在所有同步和后台导入完成后发出 `externalAgentConfig/import/completed`。这些通知包含与响应中相同的 `importId`，以及按类型列出 `successes` 和 `failures` 的 `itemTypeResults`。完成通知可能在响应之后立即到达，也可能在后台远程导入完成后到达。

```
{ "method": "externalAgentConfig/import/progress", "params": {
  "importId": "8ae96ff3-3425-4f4c-8772-b6fd61502868",
  "itemTypeResults": [
    {
      "itemType": "AGENTS_MD",
      "successes": [
        { "itemType": "AGENTS_MD", "cwd": "/Users/me/project", "source": null, "target":
"/Users/me/project/AGENTS.md" }
      ],
      "failures": []
    }
  ]
} }
{ "method": "externalAgentConfig/import/completed", "params": {
  "importId": "8ae96ff3-3425-4f4c-8772-b6fd61502868",
  "itemTypeResults": [
    {
      "itemType": "AGENTS_MD",
      "successes": [
        { "itemType": "AGENTS_MD", "cwd": "/Users/me/project", "source": null, "target":
"/Users/me/project/AGENTS.md" }
      ],
```

```
      "failures": []
    }
  ]
} }
```

读取此前已完成的导入：

```
{ "method": "externalAgentConfig/import/readHistories", "id": 65 }
{ "id": 65, "result": { "data": [
  {
    "importId": "8ae96ff3-3425-4f4c-8772-b6fd61502868",
    "completedAtMs": 1781784000000,
    "successes": [
      { "itemType": "AGENTS_MD", "cwd": "/Users/me/project", "source": null, "target":
"/Users/me/project/AGENTS.md" }
    ],
    "failures": []
  }
] } }
```

支持的 `itemType` 值包括 `AGENTS_MD`、`CONFIG`、`SKILLS`、`PLUGINS`、`MCP_SERVER_CONFIG`、`SUBAGENTS`、`HOOKS`、`COMMANDS` 和 `SESSIONS`。对于 `PLUGINS` 条目，`details.plugins` 会列出每个 `marketplaceName` 以及 Codex 可以尝试迁移的 `pluginNames`。检测只返回仍有工作要做的条目。例如，当 `AGENTS.md` 已存在且非空时，Codex 会跳过 AGENTS 迁移；技能导入也不会覆盖已有的技能目录。

从 `.claude/settings.json` 检测插件时，Codex 会从 `extraKnownMarketplaces` 读取已配置的市场源。如果 `enabledPlugins` 包含来自 `claude-plugins-official` 的插件但市场源缺失，Codex 会推断来源为 `anthropics/claude-plugins-official`。

认证端点

> ⓘ 如果你已经使用 Codex app-server 认证构建了本地或开源应用，可以继续使用，但我们建议迁移到 Sign in with ChatGPT，让用户能更好地控制和了解自己的使用情况。Sign in with ChatGPT 的开发者文档见这里，我们的实现指南（cookbook）见这里。App-server 认证从未被允许用于商业或托管服务。我们推出 Sign in with ChatGPT 就是为了支持这些用例，也很期待看到你构建的成果。请在这里加入我们的合作伙伴等候名单。

JSON-RPC 的认证/账号接口提供请求/响应方法，以及服务器主动发起的通知（无 `id`）。可用它们确定认证状态、开始或取消登录、登出、查看 ChatGPT 速率限制，以及在额度耗尽或达到使用限制时通知工作区所有者。

认证模式

Codex 支持以下认证模式。`account/updated.authMode` 显示当前生效的模式，并在可用时包含当前的 ChatGPT `planType`。`account/read` 也会报告账号和套餐详情。

- **API 密钥（`apikey`）** – 调用方通过 `type: "apiKey"` 提供 OpenAI API 密钥，Codex 会将其存储起来用于 API 请求。
- **ChatGPT 托管（`chatgpt`）** – Codex 负责整个 ChatGPT OAuth 流程，持久化 token 并自动刷新。浏览器流程用 `type: "chatgpt"` 启动，设备码流程用 `type: "chatgptDeviceCode"`。
- **ChatGPT 外部 token（`chatgptAuthTokens`）** – 实验性模式，面向已经掌管用户 ChatGPT 认证生命周期的宿主应用。宿主应用直接提供 `accessToken`、`chatgptAccountId` 和可选的 `chatgptPlanType`，并且必须在收到要求时刷新 token。
- **Amazon Bedrock** – `account/read` 将 Bedrock 账号报告为 `type: "amazonBedrock"`，并指明凭据来自 Codex 托管的 Bedrock API 密钥（`credentialSource: "codexManaged"`）还是外部 AWS 凭据链（`credentialSource: "awsManaged"`）。对于 Codex 托管的 Bedrock API 密钥，`account/updated.authMode` 使用 `bedrockApiKey`。

API 概览

- `account/read` – 获取当前账号信息；可选择刷新 token。
- `account/login/start` – 开始登录（`apiKey`、`chatgpt`、`chatgptDeviceCode` 或实验性的 `chatgptAuthTokens`）。
- `account/login/completed`（通知）– 在一次登录尝试结束（成功或出错）时发出。
- `account/login/cancel` – 按 `loginId` 取消一个进行中的托管 ChatGPT 登录。
- `account/logout` – 登出；会触发 `account/updated`。
- `account/updated`（通知）– 每当认证模式变化时发出（`authMode`：`apikey`、`chatgpt`、`chatgptAuthTokens`、`agentIdentity`、`personalAccessToken`、`bedrockApiKey` 或 `null`），并在可用时包含 `planType`。
- `account/chatgptAuthTokens/refresh`（服务器请求）– 在授权错误后请求新的外部托管的 ChatGPT token。
- `account/rateLimits/read` – 获取 ChatGPT 速率限制。
- `account/rateLimits/updated`（通知）– 每当用户的 ChatGPT 速率限制变化时发出。
- `account/sendAddCreditsNudgeEmail` – 请求 ChatGPT 就额度耗尽或达到使用限制向工作区所有者发送邮件。
- `account/rateLimitResetCredit/consume` – 使用调用方提供的 `idempotencyKey` 值消耗一次赚取的速率限制重置。
- `account/usage/read` – 获取 ChatGPT 账号的 token 活动摘要和每日分桶数据。
- `account/workspaceMessages/read` – 获取生效的工作区消息，包括可用时的通知头条。
- `mcpServer/oauthLogin/completed`（通知）– 在 `mcpServer/oauth/login` 流程结束后发出；负载包含 `{ name, threadId, loginId?, success, error? }`。较旧的服务器会省略 `loginId`。对于应用级或插件级 OAuth 流程，`threadId` 可以为 `null`。
- `mcpServer/startupStatus/updated`（通知）– 在已配置 MCP 服务器的启动状态变化时发出；负载包含 `{ threadId, name, status, error, failureReason }`。对于应用级启动，`threadId` 为 `null`。启动失败时，`failureReason: "reauthenticationRequired"` 表示已存储的 OAuth 凭据已过期且无法刷新，因此客户端应主动提供重新连接该服务器的选项。

1) 检查认证状态

请求：

```
{ "method": "account/read", "id": 1, "params": { "refreshToken": false } }
```

响应示例：

```
{ "id": 1, "result": { "account": null, "requiresOpenaiAuth": false } }
```

```
{ "id": 1, "result": { "account": null, "requiresOpenaiAuth": true } }
```

```
{
  "id": 1,
  "result": { "account": { "type": "apiKey" }, "requiresOpenaiAuth": true }
}
```

```
{
  "id": 1,
  "result": {
    "account": {
      "type": "amazonBedrock",
      "credentialSource": "codexManaged"
    },
    "requiresOpenaiAuth": false
  }
}
```

```
{
  "id": 1,
  "result": {
    "account": {
      "type": "amazonBedrock",
      "credentialSource": "awsManaged"
    },
    "requiresOpenaiAuth": false
  }
}
```

```
{
  "id": 1,
  "result": {
    "account": {
      "type": "chatgpt",
      "email": "user@example.com",
      "planType": "pro"
    },
    "requiresOpenaiAuth": true
  }
}
```

字段说明：

- `refreshToken`（布尔值）：设为 `true` 可在托管 ChatGPT 模式下强制刷新 token。在外部 token 模式（`chatgptAuthTokens`）下，app-server 会忽略该标志。
- 当 ChatGPT 账号没有邮箱地址时，`email` 为 `null`。
- `requiresOpenaiAuth` 反映当前生效的提供方；为 `false` 时，Codex 无需 OpenAI 凭据即可运行。
- 当使用由 Codex 托管的 Bedrock API 密钥时，Amazon Bedrock 报告 `credentialSource: "codexManaged"`；走外部 AWS 凭据路径时报告 `credentialSource: "awsManaged"`。这只标识所选的凭据来源，并不校验 AWS 凭据链能否解析出凭据。

2) 使用 API 密钥登录

1. 发送：

```
{
  "method": "account/login/start",
  "id": 2,
  "params": { "type": "apiKey", "apiKey": "sk-..." }
}
```

2. 预期响应：

```
{ "id": 2, "result": { "type": "apiKey" } }
```

3. 通知：

```
{
  "method": "account/login/completed",
  "params": { "loginId": null, "success": true, "error": null }
}
```

```
{
  "method": "account/updated",
  "params": { "authMode": "apikey", "planType": null }
}
```

3) 使用 ChatGPT 登录（浏览器流程）

1. 开始：

```
{
  "method": "account/login/start",
  "id": 3,
  "params": {
    "type": "chatgpt",
    "useHostedLoginSuccessPage": true,
    "appBrand": "chatgpt"
  }
}
```

默认情况下，浏览器回调成功后会重定向到本地成功页。当无需进行组织设置时，可设置 `useHostedLoginSuccessPage: true` 改用托管成功页。启用托管成功页后，`appBrand` 可以为 `"codex"` 或 `"chatgpt"`；省略或为 `null` 时默认为 `"codex"`。

```
    {
      "id": 3,
      "result": {
        "type": "chatgpt",
        "loginId": "<uuid>",
        "authUrl":
"https://chatgpt.com/...&redirect_uri=http%3A%2F%2Flocalhost%3A<port>%2Fauth%2Fcallback"
      }
    }
```

2. 在浏览器中打开 `authUrl`；app-server 会承载本地回调。
3. 等待通知：

```
    {
      "method": "account/login/completed",
      "params": { "loginId": "<uuid>", "success": true, "error": null }
    }
```

```
    {
      "method": "account/updated",
      "params": { "authMode": "chatgpt", "planType": "plus" }
    }
```

3b) 使用 ChatGPT 登录（设备码流程）

当你的客户端需要自己掌控登录环节，或者浏览器回调不够稳定时，使用该流程。

1. 开始：

```
    {
      "method": "account/login/start",
      "id": 4,
      "params": { "type": "chatgptDeviceCode" }
    }
```

```
    {
      "id": 4,
      "result": {
        "type": "chatgptDeviceCode",
        "loginId": "<uuid>",
        "verificationUrl": "https://auth.openai.com/codex/device",
        "userCode": "ABCD-1234"
      }
    }
```

2. 向用户展示 `verificationUrl` 和 `userCode`；由前端负责用户体验。
3. 等待通知：

```
{
  "method": "account/login/completed",
  "params": { "loginId": "<uuid>", "success": true, "error": null }
}
```

```
{
  "method": "account/updated",
  "params": { "authMode": "chatgpt", "planType": "plus" }
}
```

3c) 使用外部托管的 ChatGPT token 登录（`chatgptAuthTokens`）

仅当宿主应用掌管用户的 ChatGPT 认证生命周期并直接提供 token 时，才使用这一实验性模式。客户端必须先在 `initialize` 期间设置 `capabilities.experimentalApi = true`，然后才能使用该登录类型。

1. 发送：

```
{
  "method": "account/login/start",
  "id": 7,
  "params": {
    "type": "chatgptAuthTokens",
    "accessToken": "<jwt>",
    "chatgptAccountId": "org-123",
    "chatgptPlanType": "business"
  }
}
```

2. 预期响应：

```
{ "id": 7, "result": { "type": "chatgptAuthTokens" } }
```

3. 通知：

```
{
  "method": "account/login/completed",
  "params": { "loginId": null, "success": true, "error": null }
}
```

```
{
  "method": "account/updated",
  "params": { "authMode": "chatgptAuthTokens", "planType": "business" }
}
```

当服务器收到 `401 Unauthorized` 时，可能会向宿主应用请求刷新后的 token：

```
{
  "method": "account/chatgptAuthTokens/refresh",
  "id": 8,
  "params": { "reason": "unauthorized", "previousAccountId": "org-123" }
}
```

```
{ "id": 8, "result": { "accessToken": "<jwt>", "chatgptAccountId": "org-123", "chatgptPlanType":
"business" } }
```

收到成功的刷新响应后，服务器会重试原始请求。请求会在大约 10 秒后超时。

4) 取消 ChatGPT 登录

```
{ "method": "account/login/cancel", "id": 4, "params": { "loginId": "<uuid>" } }
{ "method": "account/login/completed", "params": { "loginId": "<uuid>", "success": false, "error":
"..." } }
```

5) 登出

```
{ "method": "account/logout", "id": 5 }
{ "id": 5, "result": {} }
{ "method": "account/updated", "params": { "authMode": null, "planType": null } }
```

6) 速率限制（ChatGPT）

```
{ "method": "account/rateLimits/read", "id": 6 }
{ "id": 6, "result": {
  "rateLimits": {
    "limitId": "codex",
    "limitName": null,
    "primary": { "usedPercent": 25, "windowDurationMins": 15, "resetsAt": 1730947200 },
    "secondary": null,
    "rateLimitReachedType": null
  },
  "rateLimitsByLimitId": {
    "codex": {
      "limitId": "codex",
      "limitName": null,
      "primary": { "usedPercent": 25, "windowDurationMins": 15, "resetsAt": 1730947200 },
      "secondary": null,
      "rateLimitReachedType": null
    },
    "codex_other": {
      "limitId": "codex_other",
      "limitName": "codex_other",
      "primary": { "usedPercent": 42, "windowDurationMins": 60, "resetsAt": 1730950800 },
      "secondary": null,
      "rateLimitReachedType": null
    }
  },
  "rateLimitResetCredits": {
    "availableCount": 2,
    "credits": [{
      "id": "RateLimitResetCredit_1",
      "resetType": "codexRateLimits",
      "status": "available",
      "grantedAt": 1781654400,
      "expiresAt": 1784246400,
      "title": "Rate-limit reset",
      "description": "Reset an eligible Codex rate-limit window."
    }]
  }
```

```
} }
{ "method": "account/rateLimits/updated", "params": {
  "rateLimits": {
    "limitId": "codex",
    "primary": { "usedPercent": 31, "windowDurationMins": 15, "resetsAt": 1730948100 }
  }
} }
```

字段说明：

- `rateLimits` 是向后兼容的单桶视图。
- `rateLimitsByLimitId`（存在时）是以计量 `limit_id`（例如 `codex`）为键的多桶视图。
- `limitId` 是计量桶的标识符。
- `limitName` 是该桶可选的面向用户的标签。
- `usedPercent` 是配额窗口内的当前用量。
- `windowDurationMins` 是配额窗口的时长。
- `resetsAt` 是下一次重置的 Unix 时间戳（秒）。
- 当服务器返回与某个桶关联的 ChatGPT 套餐时，会包含 `planType`。
- 当服务器返回剩余的工作区额度详情时，会包含 `credits`。
- `rateLimitReachedType` 在达到限制时标识由服务器分类的限制状态。
- `rateLimitResetCredits` 在服务提供时包含可用的赚取重置次数；否则为 `null`。
- 当只知道数量时，`rateLimitResetCredits.credits` 为 `null`。空数组表示服务已获取详情但没有可用的重置额度。服务可能会限制详情行数，因此以 `availableCount` 为准。
- 每个详情行都包含一个不透明的 `id`、`resetType`、`status`、`grantedAt`、`expiresAt`（可为 `null`）、`title`（可为 `null`）和 `description`（可为 `null`）。
- 消耗一次重置后，请再获取 `account/rateLimits/read`。

7) Token 用量（ChatGPT）

使用 `account/usage/read` 获取 ChatGPT token 活动摘要字段和可选的每日分桶数据。

```
{ "method": "account/usage/read", "id": 7 }
{ "id": 7, "result": {
  "summary": {
    "lifetimeTokens": 1234567,
    "peakDailyTokens": 45678,
    "longestRunningTurnSec": 540,
    "currentStreakDays": 8,
    "longestStreakDays": 14
  },
  "dailyUsageBuckets": [
    { "startDate": "2026-06-18", "tokens": 12345 }
  ]
} }
```

字段说明：

- 当服务尚未返回某项指标时，`summary` 中对应的值可能为 `null`。

- `dailyUsageBuckets` 可能为 `null`；存在时，每个桶都包含 `startDate` 和 `tokens`。
- 该端点需要由 Codex 服务支持的认证。ChatGPT、外部 ChatGPT token、智能体身份（agent identity）和个人访问令牌（personal access token）认证可用；仅 API 密钥认证和 Bedrock 认证不可用。

8) 赚取的速率限制重置（ChatGPT）

使用 `account/rateLimitResetCredit/consume` 消耗一次赚取的重置。

```
{ "method": "account/rateLimitResetCredit/consume", "id": 8, "params": { "idempotencyKey": "8ae96ff3-3425-4f4c-8772-b6fd61502868", "creditId": "RateLimitResetCredit_1" } }
{ "id": 8, "result": { "outcome": "reset" } }
```

字段说明：

- `idempotencyKey` 必须非空。为每次逻辑兑换尝试使用一个 UUID，并在重试该尝试时复用同一个值。
- `creditId` 是可选的。提供时，它必须是来自 `account/rateLimits/read` 的非空不透明 ID。省略时，由服务选择下一个可用的额度。
- `reset` 表示已消耗一个额度。
- `alreadyRedeemed` 表示同一兑换此前已完成。请将其视为幂等成功，并刷新账号限制。
- `nothingToReset` 表示没有符合条件的可重置速率限制窗口。
- `noCredit` 表示该账号没有可用的赚取重置额度。
- 消耗一次重置后，请获取 `account/rateLimits/read`，而不是从此响应推断更新后的窗口。

9) 向工作区所有者发送限制通知

当额度耗尽或达到使用限制时，使用 `account/sendAddCreditsNudgeEmail` 请求 ChatGPT 向工作区所有者发送邮件。

```
{ "method": "account/sendAddCreditsNudgeEmail", "id": 9, "params": { "creditType": "credits" } }
{ "id": 9, "result": { "status": "sent" } }
```

工作区额度耗尽时使用 `creditType: "credits"`；工作区达到使用限制时使用 `creditType: "usage_limit"`。如果所有者最近已被通知过，响应状态为 `cooldown_active`。

10) 工作区消息（ChatGPT）

使用 `account/workspaceMessages/read` 获取当前工作区的生效消息，包括可用时的通知头条。

```
{ "method": "account/workspaceMessages/read", "id": 10 }
{ "id": 10, "result": { "featureEnabled": true, "messages": [
  { "messageId": "msg_123", "messageType": "headline", "messageBody": "Workspace maintenance starts at 5pm.", "createdAt": 1781395200, "archivedAt": null }
] } }
```

本页为社区中文翻译，原文版权归 OpenAI 所有，以官方英文页面为准。 中文译文及排版以 CC BY-NC-SA 4.0 授权。 中英对照 Markdown（`app-server.en.md` / `app-server.zh.md`）见本仓库 `03-Codex应用服务器/` 目录。